Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)		
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

I am interested in synthetic biology and want to build a tool that can help wet-lab scientists carry out experiments. My broad goal would be to build an interactive, educational website that helps biology students and researchers find where restriction enzymes recognize and cut a DNA sequence.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

NEBcutter, WebCutter 2.0

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

I want this to be used by biology students and by anyone learning molecular biology to check their work on restriction digest problems and to understand how the results are calculated. Professional tools like NEBcutter give the answers. My tool would show the reasoning. Like where each cut happens, how fragment lengths add up to the total

sequence length, and what the analysis does and doesn't tell you about a real lab digest.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

Takes a user's DNA sequence string and selected enzyme, uses a loop to find every recognition-site position and store it in a list, and outputs the predicted fragment lengths as a table and visual map.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

The user provides a sequence name (a string, e.g., "pUC19 fragment"), a DNA sequence (a string of A, T, G, and C characters pasted into a text area), and a restriction enzyme chosen from a dropdown of predefined enzymes. For comparison mode, they also choose a second enzyme.

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

First, the program cleans the DNA sequence by removing spaces and line breaks and converting it to uppercase. It then validates it with if statements, rejecting empty input and any characters other than A, T, G, and C. Next, it looks up the selected enzyme's recognition sequence in a predefined collection of enzymes. It loops through the DNA string, compares each slice against that recognition sequence, and records the position of every match in a list. From the cut positions, it calculates fragment lengths by subtracting consecutive positions, including the start and end of the sequence, and checks that the fragments add up to the total sequence length. In comparison mode, it repeats this for a second enzyme and compares the results: the number of sites and the smallest and largest fragments.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The user gets a results page showing the selected enzyme and its recognition sequence, the number of recognition sites found, a list of their positions in the sequence, and a table of predicted fragment lengths in base pairs. The page also includes a simple visual restriction map that marks each cut site along the DNA. If no sites are found, the page says so instead of showing an empty table. In comparison mode, the user also gets a side-by-side table comparing the two enzymes, and any analysis can be saved to a history page for later review.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

DNA Sequence, a Restriction Enzyme, Restriction Analysis

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

Restriction Enzyme

Details I need: enzyme name (e.g., “EcoRI”), recognition sequence (e.g., “GAATTC”), and cut offset, meaning how many bases into the recognition sequence the enzyme actually cuts (e.g., 1 for EcoRI, which cuts after the G).
Details I will ignore: optimal temperature and buffer, sensitivity to DNA methylation, the source organism, the supplier, and whether the cut leaves sticky or blunt ends.

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

The hardest part will be making the fragment calculations correct. The cut position isn’t

always where the recognition sequence starts, so I'll need to apply each enzyme's cut offset. I'll also need to avoid off-by-one errors when converting between Python's zero-based string indexes and the one-based positions shown to users. I also need to handle edge cases correctly: sequences with no sites, overlapping sites (e.g., in repetitive sequences), and sites close to either end of the sequence. Building the visual restriction map will be challenging too, since cut positions have to be scaled so that sequences of different lengths fit in the same display width. Finally, keeping State consistent across routes without global variables will take careful planning, especially for comparison mode, which has to track two analyses at once.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will use an LLM mainly as a tutor rather than a code generator. I'll ask it for hints, explanations of concepts, and help understanding error messages, and I'll write the code myself. When I do use LLM-generated code (for example, Drafter syntax I haven't seen before), I'll only include it if I can explain every line, and I'll rewrite it in my own style instead of pasting it in. To catch hallucinations, I'll check the LLM's claims against the Drafter documentation and course materials. I'll also write unit tests with hand-calculated expected results before trusting any function, such as working out by hand that cuts at 200 and 700 in a 1,000 bp sequence give fragments of 200, 500, and 300 bp. I'll check the biology, such as recognition sequences and cut positions, against NEB's published enzyme information rather than relying on the LLM. Any technique beyond the course material will be documented in comments explaining how it works.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

My main goal is to practice combining programming with biology, which is the direction I'm interested in for research and my career. I want to build something scientifically correct that a biology student could actually use to check their restriction digest work, and to understand both the code and the molecular biology behind it well enough to explain every calculation. Along the way, I want to learn Drafter well and get better at testing, so that I can trust my program's results because I've checked them, not just because they look right.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I added the context of the constraints in my prompt too. Find me potential ideas for my web application project.

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

It mentioned generic consumer apps initially, like a schedule management tool.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

I knew I didn't want to build a generic tool but a highly specialized and technical one that suited my interests. Thus, I told it to narrow the scope to biological applications.

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

It gave me some pretty good suggestions, and I was able to explore multiple options before deciding on my idea.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

Mostly help from the LLM or office hours.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

Ask for help if stuck. If I can't solve it within a reasonable time, I'll bring the error message, what I've tried, and a minimal example to office hours or the TAs.
Isolate the problem. I'll check whether the bug is in my analysis functions or in my Drafter routes. Because my analysis functions are separate from the interface, I can test them on their own with Bakery. If those tests pass, I'll know the problem is in the route or in how State is passed around.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Final Project Roadmap

The final project will involve creating a web application with Drafter. The different steps of the project will be spread across the last several weeks of class.

- *Week 10: Brainstorming & Ideation - You will come up with an idea for your web application. You will brainstorm its goal, details, and functionality.*
- *Week 11: Design - You will plan how the site will look and how you will implement the requirements listed on this page.*
- *Week 12: Development & Deployment - You will develop the code for your website and deploy it.*
- *Week 13: User Testing - You will show your website to peers and get feedback on their experiences using it.*

- *Week 14: Updates based on Feedback - You will make updates based on the feedback you receive and reflect on the process of creating your web application.*

Site Functionality Requirements

Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- *At least 7 routes.*
- *At least 5 pages.*
- *The implementation of the State dataclass must have at least 4 fields.*
    - *At least one field must be a list of class instances or a dictionary of class instances.*
        - *This means that there must be a second dataclass.*
    - *At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.*
- *At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.*
- *At least 3 meaningful if statements.*
- *At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.*
- *The site should have a legitimate purpose or functionality.*
- *No global variables (are fine).*
- *You must use the Drafter library.*
- *Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.*

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements.

Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in . Once you are finished with your project code, check over the following:

- *Chosen good variable names (check last two sections of)*
- *Unit tested your functions where possible, being : each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).*
    - *If a function can't be unit tested, you must note as much in a docstring or comment.*
- *Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)*
- *Included types in your function headers*
- *Included good as the first line of your function's body*
- *Decomposed complicated functions with helper functions*

Your TA will be grading your code's quality based on the ideas above.

Repository Requirements

Your final project's GitHub repository should meet the following requirements.

- *Planning document should be uploaded to the repository.*
- *Website should be deployed without error.*

- *Site information should be set in `main.py` using `set_site_information(...)`.*
- *When changing/adding files in the repository, commit messages should be concise and descriptive, not the default "Update (filename)" message.*
- *A video fulfilling the following requirements should be linked in the repository.*
    - *Shows functioning deployed website*
    - *Audible and visible*
    - *Accesses and explains all necessary parts of the website*
- *All conversations with any AI should be uploaded to the repository.*

The above are either present on the final rubric or are necessary for your site to be graded. help me come up with a project idea based on these intructions

...

I recommend choosing a project that solves a problem you actually experience as a college student. The best project for this assignment will be useful enough to have a legitimate purpose, creative enough to impress your TA, and simple enough to finish in three weeks using Python and Drafter.

Your website also needs at least 7 routes, 5 pages, two dataclasses, 4 meaningfully modified state fields, 3 input fields, and a meaningful loop. I'll suggest ideas that naturally satisfy those requirements rather than forcing you to add random features.

Suggest ideas to me based on these constraints. They should be within the scope of something I can do in 3 weeks.

1. My Top 5 Project Ideas

Idea #1: Student Survival Hub 🎓

My top recommendation!

The problem

College students often struggle to keep track of assignments, exams, personal goals, and study sessions. Their information ends up scattered across notebooks, calendars, and different apps.

Your solution

Build a student dashboard where users can manage assignments, track their progress, and organize their study sessions in one place.

Features

- *Dashboard: Show the number of assignments due soon, completed tasks, and upcoming exams.*
- *Assignment Tracker: Add assignments with a course name, due date, and priority.*
- *Study Planner: Create study sessions for different courses.*
- *Progress Tracker: Mark assignments as completed and calculate completion percentages.*
- *Course Manager: Organize assignments by course.*
- *Productivity Statistics: Display a graph of completed versus incomplete assignments.*
- *Study Tips: Display suggestions based on how many assignments remain incomplete.*

Example of how it works

A student adds three assignments:

- *CISC 106: Python Project — High Priority*
- *MATH 241: Homework 5 — Medium Priority*
- *ENGL 110: Essay — Low Priority*

The dashboard displays the assignments and highlights the highest-priority task.

When the student marks the Python project complete, the dashboard updates its statistics.

How it satisfies the requirements

You can implement the project using two dataclasses.

Dataclass 1: Assignment

Field	Purpose
`title`	Assignment name
`course`	Course associated with it
`due_date`	Assignment deadline
`priority`	High, medium, or low
`completed`	Whether the assignment is finished

Dataclass 2: StudySession

Field	Purpose
`course`	Course being studied
`duration`	Study duration in minutes
`topic`	Topic covered

`completed`	Whether the session has been completed

Your `State` dataclass could contain these fields:

State field	How you modify it
`assignments`	Add assignments and update completion status
`study_sessions`	Add study sessions
`selected_course`	Update the course selected by the user
`study_goal`	Update the user's daily study goal
`current_page`	Track or update navigation state, if needed

You only need four meaningfully modified state fields, including the collection of dataclass instances. These five provide flexibility.

Example of a meaningful loop:

Loop through all assignments and display each one. Use an `if` statement to display a different message depending on whether an assignment is completed or overdue.

Possible routes and pages

Drafter routes are URL endpoints, so you can have more routes than distinct page designs.

Route	Page or function
`/`	Dashboard
`/assignments`	View all assignments
`/add-assignment`	Add an assignment
`/study-sessions`	View study sessions
`/add-session`	Add a study session
`/progress`	View productivity statistics
`/courses`	Filter assignments by course
`/complete-assignment`	Mark an assignment complete

This gives you 8 routes and 7 distinct page designs or views, depending on how you structure your application.

Difficulty

Beginner to intermediate.

Most of the functionality involves lists, dataclasses, loops, `if` statements, and input

fields—all appropriate for an introductory Python course.

You could also use browser local storage to preserve information between visits, if your course setup supports it. Without persistent storage, data may reset when the application reloads.

Why I recommend it: It's practical, easy to demonstrate, and naturally satisfies almost every programming requirement.

Idea #2: Budget Buddy 💸

A personal spending tracker for college students.

The problem

College students often have limited budgets and may not know where their money goes each month.

Your solution

Create a simple application that lets users record expenses, categorize purchases, and compare spending against a budget.

Features

- *Set a monthly spending limit.*
- *Add expenses.*
- *Categorize expenses as food, transportation, entertainment, or school supplies.*
- *Display total spending.*
- *Calculate remaining budget.*
- *Show spending by category.*
- *Display a warning when spending exceeds a selected threshold.*

Dataclasses

Expense

- `name`
- `amount`
- `category`
- `date`

BudgetCategory

- `name`
- `limit`
- `spent`

The `State` dataclass could track:

- `expenses`: a list of `Expense` objects.
- `categories`: a list of `BudgetCategory` objects.
- `monthly_budget`: the overall spending limit.
- `selected_category`: the currently selected category.
- `show_completed`: an optional display preference, if you give it a meaningful purpose.

Example

Suppose the user has a $300 monthly food budget and records:

- $12 for lunch.
- $45 for groceries.
- $18 for dinner.

The application calculates:

- Total spent: $75.
- Remaining budget: $225.

- *Percentage spent: 25%.*

It could also show a bar chart of spending by category.

Possible routes

1. *Dashboard*
2. *Add expense*
3. *View expenses*
4. *View categories*
5. *Edit budget*
6. *Spending statistics*
7. *Filter expenses*
8. *Budget tips*

Difficulty

Beginner to intermediate.

The calculations are straightforward, and the visualizations can make the finished project look impressive.

Important limitation: This should be a manual expense tracker, not a banking application. Connecting bank accounts, importing transactions automatically, or processing payments would add unnecessary complexity.

Idea #3: Campus Explorer 🗺️

A guide to interesting places around campus.

The problem

New students may not know where to study, eat, relax, or find useful campus resources.

Your solution

Build a campus guide that helps students find places based on their needs.

You could focus on the University of Delaware or design a guide for college campuses generally.

Features

- *Browse campus locations.*
- *Filter locations by category.*
- *Save favorite places.*
- *Rate locations.*
- *Search for a place by name.*
- *Display location details.*
- *Recommend places based on a user's preferences.*

Dataclasses

CampusLocation

- `name`
- `category`
- `description`
- `rating`
- `indoor`

Review

- `location_name`
- `rating`
- `comment`
- `visit_date`

Possible routes

1. *Home*
2. *Browse locations*
3. *Search locations*
4. *Location details*
5. *Favorites*
6. *Submit review*
7. *View reviews*
8. *Recommendations*

Difficulty

Beginner to intermediate, depending on how you implement the map.

The easiest version uses a predefined list of campus locations rather than an interactive map. You can display names, descriptions, and categories using ordinary Drafter components.

You could add a map later if time permits.

Important limitation: Don't make live building availability, real-time dining menus, or live campus navigation essential features. These could require external data sources or APIs that may not be available.

Idea #4: Meal Planner & Recipe Finder 🍳

A tool that helps students decide what to eat.

The problem

Students often have limited time and money to cook, and deciding what to make can be

difficult.

Your solution

Create a meal planner that recommends recipes based on available ingredients, budget, and dietary preferences.

Features

- *Browse a collection of recipes.*
- *Search by recipe name.*
- *Filter by meal type.*
- *Select available ingredients.*
- *Calculate estimated meal costs.*
- *Generate a grocery list.*
- *Save favorite recipes.*

Dataclasses

Recipe

- `name`
- `ingredients`
- `instructions`
- `estimated_cost`

GroceryItem

- `name`
- `quantity`
- `purchased`

Possible routes

1. *Home*

2. *Browse recipes*
3. *Search recipes*
4. *Recipe details*
5. *Meal planner*
6. *Grocery list*
7. *Favorite recipes*
8. *Budget-friendly recipes*

Difficulty

Beginner to intermediate.

You can create a small collection of sample recipes directly in your application. A large external recipe database or an AI-powered recipe generator is unnecessary.

The most interesting programming component would be comparing the ingredients the user has with the ingredients each recipe requires.

For example, if a user selects eggs, rice, and cheese, the application can identify recipes that use those ingredients and show how closely each recipe matches.

Idea #5: Habit Tracker & Personal Growth Dashboard 🌱

A visual tool for building better habits.

The problem

People often struggle to maintain habits because they forget to track their progress or find it difficult to see improvement over time.

Your solution

Build an application where users define habits, record daily progress, and view their

consistency.

Features

- *Create new habits.*
- *Set daily or weekly goals.*
- *Mark habits as completed.*
- *Track streaks.*
- *View completion percentages.*
- *Display progress charts.*
- *Receive encouraging messages based on progress.*

Dataclasses

Habit

- `name`
- `category`
- `goal`
- `completed_count`

HabitEntry

- `habit_name`
- `date`
- `completed`

Possible routes

1. *Dashboard*
2. *Add habit*
3. *View habits*
4. *Record daily progress*
5. *Progress statistics*

6. *Habit details*
7. *Completed habits*
8. *Weekly summary*

Difficulty

Beginner.

This is one of the simplest projects to implement correctly. The challenge is making it visually appealing and ensuring that tracking progress is meaningful.

You could add a calendar or a graph showing completion rates over time.

---

2. Which Project Is Best for Your Assignment?

Here's how I would compare the five ideas.

Criteria	*Student Survival Hub*	*Budget Buddy*	*Campus Explorer*	*Meal Planner*	*Habit Tracker*
Legitimate purpose	*Excellent*	*Excellent*	*Excellent*	*Excellent*	*Excellent*
Easy to satisfy requirements	*Excellent*	*Excellent*	*Very good*	*Very good*	*Excellent*
Interesting user interface	*Excellent*	*Excellent*	*Excellent*	*Excellent*	*Very good*

Simple Python logic	*Very good*	*Very good*	*Very good*	*Good*	*Excellent*
Opportunities for charts	*Excellent*	*Excellent*	*Optional*	*Optional*	*Excellent*
Potential usefulness after class	*Excellent*	*Excellent*	*Very good*	*Excellent*	*Very good*
Overall difficulty	*Moderate*	*Moderate*	*Moderate*	*Moderate*	*Easy*

My ranking:

1. *Student Survival Hub — best overall balance of usefulness, functionality, and manageable complexity.*
2. *Budget Buddy — best if you enjoy working with numbers, calculations, and graphs.*
3. *Campus Explorer — best if you want a visually interesting project with a campus theme.*
4. *Meal Planner — best if you enjoy food, searching, and filtering.*
5. *Habit Tracker — best if your priority is finishing a reliable application without excessive complexity.*

All five can meet the rubric. The quality of the implementation matters more than the idea alone.

---

3. My Recommendation: Build the Student Survival Hub

If I were planning this project for a CS1 student, I would choose the Student Survival Hub and make its central purpose:

Help college students organize their assignments, plan study sessions, and monitor their academic progress in one place.

The key is to build a small but complete application instead of attempting to recreate an entire productivity platform.

Your proposed five main pages

Page 1: Dashboard

The first page a user sees.

Display:

- *Total assignments.*
- *Number completed.*
- *Number remaining.*
- *Assignments due soon.*
- *Today's study goal.*
- *A list of priority assignments.*

Page 2: Assignment Manager

Let users:

- *View all assignments.*
- *Add an assignment.*
- *Select its priority.*
- *Enter a due date.*
- *Mark it as completed.*

Page 3: Study Planner

Let users:

- *Add a study session.*
- *Choose a course.*
- *Enter the topic.*
- *Enter the planned duration.*
- *Mark the session as completed.*

Page 4: Progress

Display:

- *Completed versus incomplete assignments.*
- *Total study time.*
- *Study time by course.*
- *Completion percentages.*

You could use a simple Matplotlib chart if it is supported in your Drafter environment.

Page 5: Course Explorer

Let users select a course and view:

- *Its assignments.*
- *Upcoming deadlines.*
- *Completed work.*
- *Associated study sessions.*

These pages would provide the core experience. Additional routes can handle actions such as adding assignments, completing tasks, and filtering records.

---

4. How to Meet Every Programming Requirement

This is the most important planning step. You want to ensure that your idea naturally supports the rubric.

Requirement	How your application will meet it
At least 7 routes	Create routes for the dashboard, assignment list, assignment form, study sessions, session form, progress, courses, and task completion.
At least 5 pages	Build five main views: Dashboard, Assignments, Study Planner, Progress, and Courses.
`State` dataclass with 4+ fields	Track assignments, study sessions, selected course, study goal, and other useful settings.
Second dataclass	Create `Assignment` and `StudySession`.
At least 4 modified state fields	Add assignments, add sessions, change assignment completion status, and update the study goal.
At least 3 input fields	Use a `TextBox` for a title, a `SelectBox` for priority, and a `CheckBox` for completion, along with other useful inputs.
At least 3 meaningful `if` statements	Check for empty titles, determine whether tasks are completed, and identify assignments due soon.

At least 1 meaningful loop	*Iterate through the assignment objects to display them, filter them, or calculate completion statistics.*
Legitimate purpose	*Help students organize their academic responsibilities.*
No global variables	*Store changing application data in `State`; use global constants only where appropriate.*
Use Drafter	*Build the website using Drafter routes and components.*
Document advanced techniques	*Explain any techniques beyond the course material in comments or docstrings.*

One critical detail about state

Don't just declare five fields in your `State` dataclass and leave them unchanged.

The rubric specifically requires meaningful modifications.

For example:

- *Adding an assignment changes `state.assignments`.*
- *Adding a study session changes `state.study_sessions`.*
- *Selecting a different course changes `state.selected_course`.*
- *Updating the daily goal changes `state.study_goal`.*

Also, when a user marks an assignment complete, update the relevant `Assignment` object within the list. Simply displaying a checkbox without changing the underlying data would not

be sufficient.

Before coding, verify that each route actually modifies the state it is supposed to modify.

---

5. Keep the Project Realistic for Drafter

Because Drafter runs Python in the browser using Pyodide, you can accomplish quite a lot without a traditional backend. However, you should plan around its limitations.

Features you can reasonably build

- *Adding and displaying assignments.*
- *Filtering assignments by course or priority.*
- *Calculating completion percentages.*
- *Tracking study-session durations.*
- *Displaying simple charts.*
- *Saving data using browser local storage, if supported by your setup.*
- *Building multiple pages and routes.*
- *Creating a consistent interface with navigation buttons.*

Features I would avoid initially

1. User accounts and passwords

A proper login system usually requires a backend and secure authentication. It isn't necessary for this project.

2. Shared assignments across different students

Your application doesn't have a backend database, so you shouldn't assume that different students can share a live assignment list.

3. Automatic Canvas assignment imports

This could require API access, authentication, and integration work beyond the scope of a three-week introductory project.

4. AI-generated study schedules

An advanced AI integration adds unnecessary complexity. A simple scheduling algorithm based on due dates and priorities is easier to build, explain, and test.

5. Complicated calendar integrations

Start by accepting dates as input and displaying assignments in a list. You can add a calendar-style view later if the core application works.

A note about saving data

You can investigate using browser local storage to keep assignments between visits. However, don't assume that ordinary Python variables will persist when the page reloads.

If you use local storage, test it early. If you cannot get it working, the application can still be useful during a session, but you should clearly understand that limitation.

---

6. Your Three-Week Development Plan

Your project roadmap already provides a sensible sequence. Here's how I would approach it.

Week 10: Brainstorming and Ideation

Goal: Decide what the application will do.

1. *Define the problem: students struggle to organize academic responsibilities.*
2. *Choose your primary users: college students.*

3. *Identify your main features.*
4. *Research existing solutions.*
5. *Define your two dataclasses.*
6. *Plan your `State` fields.*
7. *Identify your routes.*
8. *Complete the project pitch worksheet.*

Don't start by trying to implement every possible feature. Decide which features are essential.

Week 11: Design

Goal: Plan the application before writing most of the code.

1. *Sketch the five main pages.*
2. *Decide how users navigate between pages.*
3. *Define the fields for each dataclass.*
4. *Plan each route and its purpose.*
5. *Decide which inputs appear on each form.*
6. *Plan how assignment filtering and completion will work.*
7. *Identify how each rubric requirement will be satisfied.*

At the end of this week, you should know what each page does and what data it needs.

Week 12: Development and Deployment

Goal: Build a working application.

Implement features in this order:

1. *Create the `State`, `Assignment`, and `StudySession` dataclasses.*
2. *Build the dashboard and navigation.*
3. *Add the assignment form and assignment list.*
4. *Implement assignment completion.*

5. *Add study sessions.*
6. *Add filtering and progress calculations.*
7. *Build the remaining pages.*
8. *Add charts or other visual enhancements.*
9. *Test and deploy the application.*

Get a working version deployed before adding advanced features. A simple working application is much better than an ambitious one that fails to run.

Week 13: User Testing

Goal: Find out whether people can actually use your application.

Ask classmates to complete tasks such as:

- *Add an assignment.*
- *Find assignments for a particular course.*
- *Mark an assignment complete.*
- *Add a study session.*
- *Find their progress statistics.*

Ask them:

- *Was anything confusing?*
- *Could you find the features you needed?*
- *Did any buttons behave unexpectedly?*
- *What is one improvement you would suggest?*

Record their feedback instead of relying on memory.

Week 14: Updates Based on Feedback

Goal: Improve the application and demonstrate that it works reliably.

1. *Choose the most important feedback.*

2. *Fix bugs and confusing navigation.*
3. *Improve the interface.*
4. *Retest the affected features.*
5. *Verify that every route works.*
6. *Check that your code satisfies the rubric.*
7. *Record a video demonstrating the deployed website.*
8. *Upload your planning document and AI conversations to the repository.*

---

7. What Existing Applications Could You Research?

For the Student Survival Hub, useful prior solutions to investigate include:

- *Notion — Notion official website*
- *Todoist — Todoist official website*
- *Google Calendar — Google Calendar official website*

These tools already help people manage tasks, schedules, and personal information.

Your project doesn't need to outperform them. Instead, identify a specific problem your simplified application addresses.

For example, you could focus on showing students which assignments deserve attention first by combining due dates, priority, and completion status.

That gives your application a clear purpose beyond simply storing tasks.

---

8. Let's Narrow It Down to Your Interests

The Student Survival Hub is my strongest recommendation, but the best project is one you will enjoy building.

Answer these three questions, and I can help you settle on an idea that fits your interests

and the rubric.

1. Which category interests you most?

- *A. College life and productivity*
- *B. Money, budgeting, and spending*
- *C. Food, cooking, and meal planning*
- *D. Fitness, habits, and personal improvement*
- *E. Games, entertainment, and interactive experiences*
- *F. Campus life and finding interesting places*
- *G. Something else*

2. What is your main goal?

- *A. Make something genuinely useful.*
- *B. Make something visually impressive.*
- *C. Keep the coding relatively easy.*
- *D. Build something creative and unique.*
- *E. Balance all four.*

3. How comfortable are you with Python right now?

- *A. Basic Python: variables, `if` statements, loops, and functions.*
- *B. Comfortable with lists, dictionaries, and dataclasses.*
- *C. Comfortable with most of the course material and ready for a challenge.*

Once you've answered, I can help you choose a project and map it directly to the assignment requirements, including the two dataclasses, state fields, eight routes, and the features that will make it stand out.

i want something more technical, i am interested in biology, give me ideas related to that

If you're interested in biology and want something more technical, I'd lean toward building a scientific web application rather than a standard productivity app. You could make

something that models biological systems, analyzes data, visualizes scientific patterns, or simulates experiments.

Since you're using Python with Drafter and Pyodide, you have some interesting options: you can use libraries like NumPy, pandas, and Matplotlib (if supported in your environment) to perform calculations and visualize data directly in the browser.

Here are 7 biology-related project ideas, ranked with your assignment requirements and your interest in technical complexity in mind.

1. 🧬 BioSim: Biological Systems Simulator

My top recommendation if you want something technical, impressive, and manageable.

The concept

Build an interactive simulator that models how biological populations change over time.

For example, you could simulate:

- *Bacteria growing in a culture.*
- *A predator-prey ecosystem.*
- *Competition between two species.*
- *The spread of a non-medical, fictional infection through a simulated population.*

The user changes parameters, runs a simulation, and sees how the system behaves.

Example: Predator-prey simulator

Imagine modeling rabbits and foxes.

The user enters:

- *Initial rabbit population.*
- *Initial fox population.*
- *Rabbit reproduction rate.*
- *Fox predation rate.*
- *Simulation duration.*

Your program calculates how both populations change over time and plots the results.

For a more mathematical approach, you could use the classic Lotka–Volterra equations:

$$\frac{dR}{dt} = \alpha R - \beta RF$$

$$\frac{dR}{dt} = \alpha R - \beta RF$$

$$\frac{dF}{dt} = \delta RF - \gamma F$$

$$\frac{dF}{dt} = \delta RF - \gamma F$$

Where:

- R
- R = rabbit population.
- F
- F = fox population.
- α
- α = rabbit growth rate.
- β
- β = predation rate.
- δ
- δ = fox population growth from prey.
- γ
- γ = fox death rate.

You don't need to understand advanced differential equations to build a simplified version. You could approximate the population changes in discrete time steps using basic Python arithmetic and loops.

Pages and routes

Route	*Purpose*
/	*Introduction and simulator overview*
`/simulator`	*Enter parameters and run simulations*

`/results`	*View the latest simulation results*
`/population-graph`	*Visualize population changes*
`/compare`	*Compare two simulations*
`/saved-simulations`	*View saved simulations*
`/species`	*Learn about the modeled species*
`/about-model`	*Explain the equations and assumptions*

How it satisfies your rubric

Dataclass 1: `Simulation`

- `name`
- `rabbit_start`
- `fox_start`
- `duration`
- `growth_rate`

Dataclass 2: `PopulationPoint`

- *`time_step`*
- *`rabbit_population`*
- *`fox_population`*
- *`total_population`*

Your `State` could contain:

- *`simulations`: a list of `Simulation` objects.*
- *`population_history`: a list of `PopulationPoint` objects.*
- *`selected_simulation`: the simulation currently being viewed.*
- *`simulation_count`: the number of simulations run.*
- *`time_steps`: the number of steps in the current simulation.*

You could modify the first four fields through different routes, ensuring they genuinely change during program execution.

A meaningful loop would iterate through the simulation's population history to display values or calculate maximum and minimum populations.

What makes it impressive?

You could let users compare different scenarios, such as:

- *What happens when rabbits reproduce twice as quickly?*
- *What happens when the initial fox population increases?*
- *Under what conditions does one population collapse?*

A well-designed scientific graph and a clear explanation of the model's assumptions would make this feel like a real scientific tool.

Difficulty: 4/5. The mathematics and plotting require more thought than a standard CRUD application, but the model can be implemented using loops, lists, arithmetic, and functions.

Important: The results are predictions from a simplified mathematical model, not accurate forecasts of real ecosystems.

---

2. 🧬 DNA Lab: DNA Sequence Analyzer

Best if you are interested in genetics, molecular biology, or bioinformatics.

The concept

Build a mini bioinformatics platform that analyzes DNA sequences.

Users can paste a DNA sequence, and your application analyzes its composition and identifies interesting patterns.

For example, the input might be:

```
ATGCGCATATGCCGTA
```

Your program could calculate:

- *Sequence length.*
- *Number of adenine, thymine, cytosine, and guanine nucleotides.*
- *GC content percentage.*
- *Reverse complement.*
- *Potential start and stop codons.*

- *Positions of selected nucleotide patterns.*

Features

1. Sequence analysis

Count each nucleotide and calculate the GC percentage:

$$GC\%=G+CA+T+G+C\times100$$

$$GC\%=$$

$$A+T+G+C$$

$$G+C$$

$$\times100$$

2. Reverse complement

For a DNA sequence, replace:

- *A with T.*
- *T with A.*
- *C with G.*
- *G with C.*

Then reverse the resulting sequence.

3. Codon analysis

Divide a sequence into groups of three nucleotides and identify codons such as ATG or the standard DNA stop codons TAA, TAG, and TGA.

4. Sequence comparison

Compare two sequences and calculate the percentage of matching positions. For simplicity, you could require the sequences to have equal lengths.

5. Pattern search

Allow the user to enter a nucleotide pattern and find its positions in the sequence.

6. Visualizations

Use a bar chart to display nucleotide counts and a second chart to compare the composition of two sequences.

Pages and routes

Route	*Purpose*
/	*DNA Lab homepage*
/analyze	*Analyze a DNA sequence*

`/composition`	Display nucleotide statistics
`/reverse-complement`	Generate a reverse complement
`/codons`	Explore codons
`/compare`	Compare two sequences
`/pattern-search`	Find a nucleotide pattern
`/history`	View previous analyses

Dataclasses

`DNASequence`

- `name`
- `sequence`
- `length`
- `gc_content`

`AnalysisResult`

- `sequence_name`

- *`a_count`*
- *`t_count`*
- *`c_count`*
- *`g_count`*

Your `State` could hold:

- *`sequences`: a list of `DNASequence` objects.*
- *`analysis_results`: a list of `AnalysisResult` objects.*
- *`current_sequence`: the sequence being analyzed.*
- *`comparison_sequence`: the second sequence.*
- *`search_pattern`: the pattern to find.*

The first two fields can be modified as sequences are analyzed and results are stored. The other fields can change as users enter new inputs.

What makes it impressive?

You could make the application feel like a real bioinformatics tool, with a sequence input area, a nucleotide composition graph, color-coded DNA letters, and a results dashboard.

You could also include a built-in sample sequence so users can experiment immediately.

Difficulty: 3/5. This is very achievable with introductory Python, but the biological context makes it feel technical. Most operations involve string manipulation, loops, conditions, and arithmetic.

One scientific limitation: Finding a start codon or calculating GC content does not establish that a sequence is a functional gene. Your application should describe these as basic

sequence analyses, not definitive biological interpretations.

---

3. 🦠 MicrobeLab: Bacterial Growth Simulator

Best if you like microbiology and experimental science.

The concept

Build a simulator that models how a bacterial population grows over time under different environmental conditions.

The user chooses model parameters and sees how the population changes.

Features

- *Set an initial bacterial population.*
- *Adjust the growth rate.*
- *Set a carrying capacity.*
- *Simulate growth over a selected time period.*
- *Compare two simulated conditions.*
- *Plot population size against time.*
- *Display doubling time under exponential growth assumptions.*
- *Explain the lag, exponential, stationary, and death phases of bacterial growth.*

Example

Suppose you start with a hypothetical population of 100 bacteria.

Under an idealized exponential growth model:

$$N(t)=N_0e^{rt}$$

$N(t)=N$

0

e

rt

Where:

- $N(t)$
- $N(t)$ *is the population at time*
- t
- t.
- $N0$
- N
- 0
-
- *is the initial population.*
- r
- r *is the modeled growth rate.*

Your program could compare a low-growth-rate scenario with a high-growth-rate scenario and display both curves.

You could eventually introduce a logistic growth model that accounts for limited resources.

Potential routes

1. Home
2. New experiment
3. Growth simulation
4. Growth graph
5. Compare experiments
6. Experiment history
7. Growth calculator
8. Model explanation

Dataclasses

`GrowthExperiment`

- `name`
- `initial_population`
- `growth_rate`
- `duration`

`GrowthMeasurement`

- `time`
- `population`
- `experiment_name`

A loop can calculate the population at each time step and store the results in a list.

What makes it impressive?

The user can explore how changing one parameter affects an entire population trajectory.

You could display multiple curves on the same graph.

This is essentially a small computational biology application.

Difficulty: 3.5/5. The mathematics is manageable, although you'll need to decide how to handle very large population values and validate user inputs.

Important: Present this as a mathematical simulation. Avoid implying that the model predicts the behavior of a specific real microorganism without appropriate experimental data and assumptions.

---

4. 🧠 NeuroLab: Neuron Signal Simulator

Best if you like neuroscience, physiology, or electrical signals in biological systems.

The concept

Build an interactive educational simulator that demonstrates how a simplified neuron responds to stimulation.

A neuron communicates using changes in its membrane potential. Your application could show a simplified action potential waveform and let the user change the stimulus parameters.

Features

- *Adjust stimulus strength.*
- *Adjust the firing threshold.*
- *Change the simulated time interval.*
- *Display a membrane-potential graph.*

- *Identify depolarization and repolarization.*
- *Compare responses under different parameter settings.*
- *Save simulations for comparison.*
- *Display educational explanations of the different phases.*

How it works

You could implement a simplified threshold-based model:

1. *Generate a series of time steps.*
2. *Calculate the simulated membrane potential.*
3. *Check whether the potential crosses the chosen threshold.*
4. *Generate a simplified spike when the threshold is reached.*
5. *Plot the results.*

This would be an educational approximation rather than a complete biophysical model of a neuron.

Possible routes

1. *Home*
2. *Neuron simulator*
3. *Stimulus settings*
4. *Signal graph*
5. *Compare simulations*
6. *Saved experiments*
7. *Neuron anatomy*
8. *Model explanation*

Dataclasses

`NeuronSimulation`

- `name`
- `stimulus_strength`
- `threshold`
- `duration`

`SignalPoint`

- `time`
- `membrane_potential`
- `phase`

What makes it impressive?

The application could visually resemble a neuroscience lab tool, with adjustable parameters, interactive controls, and signal plots.

It offers a stronger technical identity than a standard data-entry application.

Difficulty: 4/5. Generating and plotting the waveform is manageable, but choosing a biologically reasonable model and explaining its limitations requires care.

---

5. 🌿 EcoSim: Ecosystem and Biodiversity Analyzer

Best if you like ecology, evolution, and environmental science.

The concept

Build an application that analyzes a hypothetical ecosystem containing several species.

Users can explore how species populations change and how biodiversity differs between environments.

Features

- *Create a simulated ecosystem.*
- *Add species and their population sizes.*
- *Adjust reproduction or mortality rates.*
- *Track populations over time.*
- *Compare two ecosystems.*
- *Calculate species richness.*
- *Display population distributions.*
- *Visualize how biodiversity changes.*

You could start with simple population models and later add interactions between species.

Dataclasses

`Species`

- `name`
- `population`
- `growth_rate`
- `environment`

`EcosystemSnapshot`

- `time_step`
- `species_populations`
- `species_count`

Possible routes

1. *Home*
2. *Create ecosystem*
3. *Add species*
4. *Current populations*
5. *Simulate ecosystem*
6. *Compare ecosystems*
7. *Biodiversity statistics*
8. *Model explanation*

What makes it impressive?

You can build an application that looks like a scientific ecology dashboard, with population charts and biodiversity statistics.

You could also demonstrate how removing one species affects a simplified food web, although modeling these interactions accurately adds complexity.

Difficulty: 4/5. The basic statistics are easy; modeling interactions among species is the challenging part.

---

6. 🧪 LabBench: Scientific Experiment Analyzer

Best if you want something that resembles a real research tool.

The concept

Create a general-purpose scientific data analysis application for biology experiments.

A researcher or student enters experimental measurements, and the program calculates summary statistics and visualizes the results.

For example, you could use simulated data from an experiment examining plant growth under different light conditions.

Features

- *Create an experiment.*
- *Add observations.*
- *Group measurements by treatment.*
- *Calculate averages and ranges.*
- *Compare experimental groups.*
- *Plot results.*
- *Identify the maximum and minimum observations.*
- *Save experiments and view previous results.*

Example

Suppose you have three hypothetical plant-growth groups:

Group	*Heights (cm)*
Low light	*4, 5, 6, 4, 5*

Medium light	*7, 8, 9, 7, 8*
High light	*6, 7, 5, 8, 6*

Your application calculates the average for each group and creates a bar chart.

This lets users explore whether the groups appear to differ.

Dataclasses

`Experiment`

- *`name`*
- *`organism`*
- *`treatment`*
- *`description`*

`Measurement`

- *`experiment_name`*
- *`sample_id`*
- *`value`*
- *`unit`*

Possible routes

1. *Home*
2. *Create experiment*

3. *Add measurement*
4. *View measurements*
5. *Summary statistics*
6. *Compare groups*
7. *Data visualization*
8. *Experiment history*

What makes it impressive?

This project is less about simulating biology and more about building a scientific computing application.

You could add CSV upload support, which would let users analyze data rather than manually entering every measurement. Start with manual input, however, so that file processing doesn't become a blocker.

Difficulty: 3.5/5. The statistics and graphs are straightforward, but organizing experimental data carefully takes some planning.

Scientific limitation: Differences between group averages alone do not prove that a treatment caused an effect. You should present descriptive statistics as summaries rather than definitive conclusions.

---

7. 🧬 EvolutionLab: Natural Selection Simulator

Best if you like evolutionary biology and computational modeling.

The concept

Create a simulated population of organisms with different inherited traits and model how the trait distribution changes over generations.

For example, imagine a hypothetical beetle population containing green and brown beetles.

In a green environment, the model assigns green beetles a higher survival probability. The user runs the simulation for several generations and observes changes in the population.

Features

- *Set the initial population.*
- *Select the frequency of different traits.*
- *Adjust survival probabilities.*
- *Set the number of generations.*
- *Simulate reproduction and selection.*
- *Plot trait frequencies over time.*
- *Compare two environments.*
- *Explain natural selection.*

Dataclasses

`Organism`

- *`trait`*
- *`generation`*
- *`survival_probability`*

`GenerationSummary`

- *`generation_number`*

- *green_count*
- *brown_count*
- *total_population*

Possible routes

1. *Home*
2. *Configure population*
3. *Run evolution*
4. *Generation results*
5. *Trait-frequency graph*
6. *Compare environments*
7. *Saved simulations*
8. *Evolution concepts*

What makes it impressive?

You could demonstrate an evolutionary process over multiple generations rather than simply displaying facts about evolution.

The application could include a graph showing the percentage of each trait over time.

Difficulty: 4/5. The model can be relatively simple, but random sampling, population updates, and explaining the assumptions require care.

You should make clear that the outcome is one possible simulation, not a universal prediction of evolution in real populations.

---

8. Comparison: Which Is the Best Fit?

Project	Main technical skill	Biology area	Difficulty	Visual potential
BioSim	Mathematical modeling	Ecology	4/5	Excellent
DNA Lab	String processing and algorithms	Genetics / bioinformatics	3/5	Excellent
MicrobeLab	Growth modeling	Microbiology	3.5/5	Excellent
NeuroLab	Signal generation and modeling	Neuroscience	4/5	Excellent
EcoSim	Population modeling and statistics	Ecology	4/5	Excellent
LabBench	Data analysis and statistics	Experimental biology	3.5/5	Excellent
EvolutionLab	Simulation and probability	Evolutionary biology	4/5	Excellent

These difficulty ratings are estimates, assuming you're comfortable with basic Python and have access to the relevant libraries.

My personal top three for you

🥇 *First: DNA Lab*

Choose this if you want a technical project with a clear biological purpose, interesting algorithms, and manageable complexity.

Why I like it:

- *It uses real concepts from bioinformatics.*
- *The calculations are deterministic and easy to test.*
- *It naturally supports several pages and routes.*
- *It requires meaningful string processing, loops, and conditionals.*
- *It can have a professional scientific interface.*
- *You can add more advanced features without making the core project too difficult.*

🥈 *Second: BioSim*

Choose this if you want the project to feel more like computational biology.

Why I like it:

- *You can explore biological behavior by changing parameters.*
- *The math and graphs make the application feel technical.*
- *Comparing simulations gives you an interesting feature that goes beyond simple data entry.*

- *It demonstrates computational thinking and scientific modeling.*

Its main drawback is that you need to understand and explain the model's equations and assumptions.

🥉 *Third: NeuroLab*

Choose this if neuroscience interests you most.

Why I like it:

- *It has an interesting scientific subject.*
- *It provides opportunities for adjustable controls and dynamic visualizations.*
- *The results are easy to demonstrate in a video.*
- *It could look like a miniature neuroscience research tool.*

Its main drawback is that a biologically credible model requires more care than a simple DNA analyzer.

---

9. An Idea That Combines the Best of Several Projects

If you want something particularly ambitious but still manageable, you could create BioLab: An Interactive Biology Computing Toolkit.

Instead of trying to build one enormous simulator, you could build a unified scientific application with three modules:

1. *Sequence Analyzer: Analyze DNA composition and search for nucleotide patterns.*
2. *Population Simulator: Simulate biological population changes over time.*
3. *Experiment Analyzer: Calculate statistics and plot experimental measurements.*

Each module would have its own pages, inputs, and results.

This would be a compelling demonstration of scientific programming, but it also has a risk: three separate modules mean more code, more testing, and more opportunities for bugs.

For a three-week CS1 project, I would only choose this approach if you keep each module very small and finish one before beginning another.

My recommendation is to choose one biological problem and solve it well. A polished DNA analyzer or population simulator is likely to be stronger than three unfinished tools.

Let's narrow it down

I have three questions that will help me identify the strongest project for you:

1. Which area of biology interests you most?

- *A. Genetics, DNA, and molecular biology*
- *B. Neuroscience and the brain*
- *C. Microbiology, bacteria, and viruses*
- *D. Ecology, ecosystems, and evolution*
- *E. Human biology, physiology, and health*
- *F. Biology in general; I like the computational side most*

2. What kind of technical challenge sounds most interesting?

- *A. Algorithms that analyze biological sequences or data.*
- *B. Mathematical models and simulations.*

- *C. Graphs, statistics, and scientific data analysis.*
- *D. A combination of all three.*

3. How ambitious do you want to be?

- *A. Moderate: polished, technical, and relatively safe to finish.*
- *B. Challenging: something that stands out from typical CS1 projects.*
- *C. Very ambitious: willing to take on more complex biology and programming.*

Your answers will help me narrow this down to one specific project concept with a clear purpose, technical features, and a realistic implementation plan that satisfies every major rubric requirement.

for the DNA lab, i want to be adressing a specific problem that is faced by scientists that my tools can solve, what are other tools doing and what are some applications i can focus on

DNA Lab: Turning a Bioinformatics Tool into a Solution for Scientists

I think this is a much stronger direction for your project. Instead of building a generic DNA analyzer that calculates GC content and generates reverse complements, you can identify a specific problem that researchers encounter when working with DNA sequences and build a lightweight tool that addresses it.

The key is to choose a problem that:

1. *Scientists genuinely encounter.*
2. *Existing software already attempts to solve, so you can research how your tool could be different.*
3. *You can meaningfully address with Python, without needing a biological database or a powerful backend.*

4. *Allows you to demonstrate algorithms, data structures, visualization, and scientific reasoning.*
5. *Can be implemented and tested within your three-week development period.*

One important distinction: you don't need to invent an entirely new bioinformatics problem. You can build a simpler, more focused solution for a particular type of user or workflow.

Here are the directions I would investigate first.

1. What problems do scientists actually face?

There are several different stages in DNA research, and each creates different computational problems.

Consider a simplified workflow for a molecular biology researcher:

1. *Obtain a DNA sequence. A sequencing experiment produces data that needs to be processed.*
2. *Check the sequence. Researchers need to identify potential quality problems, unexpected characters, and other issues.*
3. *Compare sequences. They may want to determine how an experimental sequence differs from an expected reference.*
4. *Interpret the sequence. They may look for genes, coding regions, or particular DNA motifs.*
5. *Record and communicate results. They need to document findings so they can be reviewed and reproduced.*

Each step presents an opportunity for a tool. However, existing software already handles many of these tasks, so your project needs a clearly defined scope.

Here are the main tools worth researching and what they tell us about potential project opportunities.

2. What existing bioinformatics tools do

A. NCBI BLAST — Finding similar DNA sequences

Official website: NCBI BLAST

The problem it solves: A researcher has a DNA sequence but wants to find similar sequences in a biological database.

For example, a researcher might have a sequence from an environmental sample and want to investigate which known organisms have similar sequences.

BLAST searches biological sequence databases and identifies regions of similarity. It is widely used for sequence identification and investigating evolutionary or functional relationships.

What your tool could learn from it:

- *How to compare sequences.*
- *How to present matching regions.*
- *How to summarize similarity using understandable statistics.*

What I would not attempt: Rebuilding BLAST. Its database searching, alignment heuristics, and statistical significance calculations are far beyond what you need for this project.

Instead, you could compare a small set of sequences provided by the user, without

searching an external database.

B. Benchling — Managing and analyzing DNA designs

Official website: Benchling Molecular Biology

The problem it solves: Molecular biologists need to design, inspect, analyze, and manage DNA sequences as part of research workflows.

Benchling offers features for sequence visualization, annotations, alignments, primer design, and other molecular biology tasks.

What your tool could learn from it:

- *Organizing sequence information.*
- *Showing annotations and sequence features.*
- *Making analysis results easier to understand visually.*
- *Keeping a record of analyses.*

The opportunity: Rather than building an entire molecular biology platform, focus on one narrow workflow that you can implement particularly clearly.

For example, your application could take a reference sequence and an experimental sequence and produce an easy-to-read discrepancy report.

That would be a small, focused application rather than a replacement for Benchling.

C. NCBI ORFfinder — Identifying potential coding regions

Official website: NCBI ORFfinder

The problem it solves: Researchers sometimes need to identify regions of a DNA sequence that could encode proteins.

ORFfinder identifies open reading frames (ORFs), reports their coordinates, and can provide their translated protein sequences.

What your tool could learn from it:

- *Searching for patterns in DNA.*
- *Identifying start and stop codons.*
- *Calculating positions within a sequence.*
- *Presenting potential coding regions.*

The opportunity: You could make an educational ORF exploration tool that shows how the three possible reading frames on each strand affect the results.

However, finding ORFs is an established task, so this idea would be most compelling if you had a specific audience or a distinct visualization approach.

D. FastQC — Assessing sequencing-read quality

Official website: FastQC project

The problem it solves: Researchers working with high-throughput sequencing data need to assess whether their sequencing reads exhibit potential quality problems before

downstream analysis.

FastQC generates quality-control reports for sequencing data. Tools in this workflow can help identify issues such as adapter contamination and low-quality reads.

What your tool could learn from it:

- *Checking many sequences systematically.*
- *Flagging potential problems rather than simply displaying raw data.*
- *Generating a report that prioritizes issues for review.*
- *Visualizing statistics across a collection of sequences.*

The opportunity: Build a lightweight sequence-quality screening tool for a small collection of DNA sequences.

One important distinction: FastQC analyzes sequencing-read data, including per-base quality information. If your input contains only plain DNA letters, you cannot reproduce those quality measurements. Your application would instead assess properties such as sequence length, nucleotide composition, ambiguous bases, and differences from an expected reference.

This is a more realistic scope for a browser-based CS1 project.

---

3. Five specific problems you could focus on

Now let's turn these observations into project ideas with concrete scientific purposes.

Idea 1: SequenceCheck — Finding unexpected differences in DNA

sequences

My favorite option for a technically focused project.

The scientific problem

Researchers may need to check whether an experimentally obtained DNA sequence matches an expected reference sequence.

For example, suppose a researcher expects a particular DNA fragment but obtains this sequence after an experiment:

Position	Expected sequence	Observed sequence
1–5	ATGCA	ATGCA
6–10	GTTAC	GTCAC
11–15	CAGTA	CAGTA

The sequences differ at one position: the seventh nucleotide.

Finding such differences by manually inspecting long sequences can be tedious and error-prone.

Your solution

Build SequenceCheck, a tool that compares an expected DNA sequence against an observed sequence and generates a discrepancy report.

The user enters the reference sequence and the observed sequence.

Your application:

1. *Validates both sequences.*
2. *Checks whether they have the same length.*
3. *Compares corresponding positions.*
4. *Identifies mismatches.*
5. *Calculates the number and percentage of matching positions.*
6. *Displays a report showing where the sequences differ.*

For example:

Comparison report

- *Reference length: 15 nucleotides.*
- *Observed length: 15 nucleotides.*
- *Matching positions: 14.*
- *Mismatches: 1.*
- *Percentage of matching positions: 93.33%.*
- *Mismatch location: Position 7, reference G, observed C.*

You could color-code the sequences to highlight mismatches.

What existing tools already do

BLAST identifies regions of sequence similarity, while Benchling provides more

comprehensive sequence analysis and alignment capabilities.

Your application would not offer their full functionality. It would instead provide a simple, transparent comparison workflow for short sequences.

What would make your version useful?

Focus on explaining discrepancies rather than merely reporting a similarity score.

For example:

- *Show exactly where the sequences differ.*
- *Separate matches from mismatches visually.*
- *Explain what a nucleotide substitution is.*
- *Let users compare multiple observed sequences against the same reference.*
- *Produce a summary table that can be downloaded as a CSV file.*

This last feature is particularly useful for demonstrating data processing.

The important limitation

For the initial version, require equal-length sequences and compare them position by position. If the lengths differ, explain that the basic comparison mode cannot reliably identify all corresponding positions.

Insertions and deletions require more sophisticated sequence alignment. You could investigate adding a basic alignment algorithm later, but it should not be necessary for the first version.

Also, a mismatch does not automatically mean a biological mutation occurred. It could

reflect a sequencing error, a sample difference, or another cause that requires investigation.

Technical difficulty: 3/5.

You would use string processing, loops, conditional statements, lists, and simple calculations. Those are excellent skills to demonstrate in a CS1 project.

Idea 2: AmpliconQC — Screening DNA sequences for potential quality issues

Best if you want to focus on quality control.

The scientific problem

Before researchers use DNA sequences in downstream analyses, they may need to check whether the data contains unexpected characters, unusually short sequences, ambiguous nucleotides, or other properties that warrant further review.

A collection of sequences makes these checks more tedious because the researcher must inspect multiple records.

Your solution

Build a tool that accepts several DNA sequences and produces a quality-control report.

For each sequence, your application could calculate:

Metric	*What it tells the user*

Sequence length	*How many nucleotides are present*
GC content	*Percentage of nucleotides that are G or C*
Ambiguous bases	*How many positions contain symbols such as N*
Invalid characters	*Whether the input contains symbols outside the accepted alphabet*
Repeated sequences	*Whether multiple records contain identical sequences*
Length outliers	*Whether a sequence differs substantially from the collection's typical length*

Your application could flag sequences for review rather than declaring them biologically invalid.

For example, if the user provides ten sequences and one contains unexpected characters, the report highlights that record.

What existing tools already do

FastQC and other sequencing-quality tools address quality assessment for sequencing

data, while other tools perform sequence preprocessing and filtering.

Your version would be much simpler. It would analyze the nucleotide strings themselves rather than claiming to evaluate raw sequencing quality.

A useful project distinction

Instead of trying to replicate a comprehensive sequencing-quality pipeline, focus on a quick screening report for a small collection of DNA sequences.

You could also explain why a particular property deserves review.

For example:

- *A sequence containing many ambiguous bases might be difficult to interpret.*
- *A sequence much shorter than the others might deserve inspection.*
- *Identical sequences might be intentional or might indicate duplicate records.*

These are flags for human review, not automatic proof of an error.

Technical difficulty: 3/5.

This idea is a good fit for lists, loops, dataclasses, input validation, statistics, and charts.

Idea 3: RestrictionMap — Understanding where DNA can be cut

Best if you want a tool that resembles something used in molecular biology laboratories.

The scientific problem

Researchers working with plasmids and other DNA molecules may need to determine where particular restriction enzymes recognize and cut a DNA sequence.

These enzymes recognize particular nucleotide patterns. Locating those patterns helps researchers understand the possible fragments produced by restriction digestion.

Your solution

Build a tool that scans a DNA sequence for recognition sites associated with a small, predefined collection of restriction enzymes.

For example, suppose a user enters a sequence and selects an enzyme that recognizes `GAATTC`*.*

Your program searches for every occurrence of that pattern and reports its position.

The user could then:

- *View the locations of recognition sites.*
- *Compare the sites recognized by different enzymes.*
- *See how many sites occur in the sequence.*
- *Estimate fragment lengths under simplified digestion assumptions.*
- *Export a restriction-site report.*

What existing tools do

The EMBOSS `restrict` *and* `remap` *tools already identify restriction sites and can report their locations. Some options also report potential fragment lengths.*

You would not be inventing this functionality. Instead, you could build a simpler, highly visual tool for exploring how recognition patterns relate to DNA sequences.

A possible differentiator

Create an interactive-looking sequence map that displays the DNA sequence and marks each recognition site.

You could also let the user select two enzymes and compare their recognition patterns.

Difficulty: 3/5. Searching for patterns and reporting positions is manageable with basic Python.

One limitation: real restriction digestion can depend on factors beyond the recognition sequence, including methylation and enzyme-specific behavior. Your initial version should clearly describe its simplified assumptions.

Idea 4: PrimerCheck — Comparing candidate PCR primers against a target sequence

Best if you want to focus on a specific molecular biology workflow.

The scientific problem

Researchers use PCR primers to target particular DNA regions. A primer that matches the wrong location or has unsuitable properties may not behave as intended.

Choosing appropriate primers and assessing their specificity are important parts of

experimental planning.

Your solution

Create a basic tool that evaluates candidate primer sequences against a user-provided DNA template.

Potential features:

- *Validate primer sequences.*
- *Calculate primer length.*
- *Calculate GC percentage.*
- *Estimate melting temperature using a clearly documented simplified formula.*
- *Search for exact matches and reverse-complement matches within the template.*
- *Report candidate binding positions.*
- *Compare several candidate primers.*

What existing tools do

The NCBI Primer-BLAST combines primer-design functionality with database-based specificity checking. It evaluates potential matches against selected biological databases and is substantially more sophisticated than a simple sequence scanner.

What your version could focus on

Rather than designing experimentally validated primers, you could create an educational primer inspection and reporting tool.

It would explain why each property matters and show where a candidate sequence matches the supplied template.

Difficulty: 3.5/5.

The basic calculations are manageable, but accurate primer evaluation involves more biology than GC content and exact matching. Your tool should not claim that a candidate is experimentally suitable or specific to an organism without appropriate analysis.

Idea 5: VariantViewer — Visualizing differences among DNA sequences

Best if you enjoy algorithms and data visualization.

The scientific problem

Researchers may need to compare multiple DNA sequences to investigate how they differ.

For example, a collection of sequences from different samples might contain substitutions at several positions. Finding those differences helps researchers describe variation within the collection.

Your solution

Create a tool that accepts a reference sequence and multiple observed sequences.

It could:

- *Compare each sequence against the reference.*
- *Count matching and mismatching positions.*
- *Identify positions where the samples differ.*
- *Calculate the frequency of each nucleotide at a position.*
- *Generate a table of observed differences.*

- *Visualize sequence variation using a heatmap or another simple chart.*

Example

Suppose three sample sequences differ from a reference at the same position.

Your tool could show that two samples contain C *at position 25, while one contains* T*.*

This is a description of the supplied sequences, not proof that the difference has a biological effect.

What existing tools do

BLAST focuses on identifying sequence similarities, while Benchling offers sequence alignment and visualization features.

Your project could focus specifically on making differences among a small collection of sequences easy to inspect.

For your initial version, require equal-length sequences and compare corresponding positions. A full alignment tool would require more sophisticated algorithms.

Difficulty: 3.5/5. This has more analytical depth than a basic sequence analyzer, especially if you add nucleotide-frequency calculations and visualization.

---

4. Which idea addresses the strongest specific problem?

Here is how I would compare them for your project.

Project	Specific scientific problem	Main technical challenge	Main limitation
SequenceCheck	Identifying discrepancies between expected and observed sequences	Comparing sequences and reporting mismatches	Basic version assumes equal lengths
AmpliconQC	Screening a collection of sequences for potential quality issues	Validation, statistics, and outlier detection	Cannot assess raw sequencing quality without quality data
RestrictionMap	Finding restriction-enzyme recognition sites	Pattern searching and position calculations	Limited enzyme collection
PrimerCheck	Inspecting candidate primers against a supplied template	Sequence matching and primer calculations	Cannot establish full primer specificity
VariantViewer	Summarizing variation across multiple sequences	Position-by-position comparison and visualization	Basic version does not handle insertions and deletions

All five are realistic, but I would prioritize the first, second, and fifth because they combine meaningful scientific data processing with features you can implement and test in Python.

5. My recommendation: Build SequenceCheck with a quality-control component

I would combine the most useful aspects of SequenceCheck and AmpliconQC into one focused application.

The project would not simply be a DNA analyzer. Its purpose would be:

Help researchers inspect a small collection of DNA sequences, identify potential data-quality issues, and compare observed sequences against an expected reference.

That gives you a specific workflow to investigate, a clear user, and measurable outputs.

You could call it SeqAudit: DNA Sequence Quality and Comparison Tool.

The problem statement

Here's the central problem I'd propose investigating:

Researchers who work with short DNA sequences may need to validate their input data and compare observed sequences against an expected reference. Performing these checks manually can be tedious, especially when several sequences must be reviewed. A lightweight tool that automatically checks sequence formatting, summarizes basic sequence properties, and highlights differences from a reference could make this initial inspection more convenient.

Notice the wording: you're proposing to make a workflow more convenient, not claiming that scientists lack existing software for it.

The target user

Your initial target user could be:

- *An undergraduate biology student working with DNA sequence data.*
- *A researcher who needs to inspect a small set of sequences.*
- *A teaching laboratory instructor who wants students to explore sequence differences.*
- *A bioinformatics beginner learning how to interpret sequence-comparison results.*

For a CS1 project, I would focus on students and researchers working with small sequence collections, rather than attempting to support large-scale sequencing pipelines.

The core workflow

Imagine a user has five DNA sequences and wants to compare them against a reference.

They would:

1. *Paste the reference sequence.*
2. *Enter or upload the observed sequences.*
3. *Run basic input validation.*
4. *View sequence lengths, GC content, and ambiguous-base counts.*
5. *Compare equal-length sequences against the reference.*
6. *View the positions where mismatches occur.*
7. *Download a summary report.*

This is a complete workflow with a clear beginning and end.

---

6. What would make SeqAudit technically interesting?

I would build it in three layers.

Layer 1: Sequence validation

The application checks whether the supplied sequences can be analyzed.

Possible checks include:

- *Empty input.*
- *Invalid characters.*
- *Sequence length.*
- *Ambiguous nucleotide symbols.*
- *Duplicate sequences.*

For example, if a user enters `ATGCXZ`, your program can report that `X` and `Z` are not accepted in the application's basic DNA alphabet.

You could optionally support `N`, which represents an unknown or ambiguous nucleotide, while explaining how it affects your calculations.

Layer 2: Sequence comparison

The user selects a reference sequence and compares one or more observed sequences against it.

For equal-length sequences, your program can calculate:

$$\text{Identity}(\%)=\text{Number of matching positions}\text{Sequence length}\times 100$$

$$\text{Identity}(\%)=$$

$$\text{Sequence length}$$

$$\text{Number of matching positions}$$

$$\times 100$$

Your application would also report:

- *The total number of matching positions.*
- *The number of mismatches.*
- *The mismatch positions.*
- *The reference and observed nucleotide at each mismatch.*

For example, if 98 of 100 positions match, the positional identity is 98%.

This simple measure is appropriate for equal-length, position-by-position comparisons. It is not a substitute for general sequence alignment, and it should not be used to infer biological relatedness on its own.

Layer 3: Results and visualization

Display the results in a scientific dashboard.

Potential visualizations include:

- *A bar chart of sequence lengths.*
- *A bar chart of GC content.*
- *A table highlighting mismatch positions.*
- *A summary of how many sequences passed each validation check.*

You could use Matplotlib if it works in your Drafter environment.

This layer would make the project visually impressive without requiring complicated biological algorithms.

---

7. How you could satisfy the project rubric

Your application naturally supports the required dataclasses and state fields.

Dataclass 1: `DNASequence`

Field	*Purpose*
`name`	*Identifies the sequence*
`sequence`	*Stores the nucleotide string*
`length`	*Stores its length*
`gc_content`	*Stores its calculated GC percentage*
`ambiguous_count`	*Stores the number of ambiguous bases*

Dataclass 2: `SequenceComparison`

Field	Purpose
`reference_name`	Identifies the reference
`sample_name`	Identifies the observed sequence
`match_count`	Number of matching positions
`mismatch_count`	Number of mismatching positions
`identity_percentage`	Percentage of matching positions

Suggested `State` fields

- `sequences`: a list of `DNASequence` objects.
- `comparisons`: a list of `SequenceComparison` objects.
- `reference_sequence`: the currently selected reference.
- `analysis_count`: number of analyses performed.
- `filter_mode`: the currently selected result filter.

You would need to ensure that at least four state fields are meaningfully modified by routes.

For example, adding sequences changes `sequences`, comparing sequences adds to `comparisons`, selecting a reference changes `reference_sequence`, and running an analysis updates `analysis_count`.

Suggested routes

Route	Purpose
`/`	Project overview and dashboard
`/input`	Enter a DNA sequence
`/collection`	View stored sequences
`/quality-check`	Validate sequences and calculate statistics
`/compare`	Select sequences to compare
`/comparison-results`	Display comparison results
`/visualization`	Display charts and summary statistics

`/history`	Review previous analyses

You could implement all of these with Drafter, lists, dataclasses, and functions. You would not need a database or external API for the core functionality.

---

8. One final consideration: What does your tool do better than existing software?

This is the most important question to investigate before committing to the idea.

Benchling, BLAST, and other bioinformatics tools already perform sophisticated sequence analysis. Your project should not claim that your application is more accurate or more powerful.

Instead, identify a narrower value proposition.

For example:

SeqAudit is a lightweight educational tool that combines basic DNA sequence validation, reference comparison, and visual reporting in a single, easy-to-understand workflow for small sequence collections.

You can investigate whether this combination is useful for your intended users. You do not need to prove that no existing software has the same features; you need to explain why your particular scope is worthwhile.

To support your planning document, I would research three questions:

1. *What sequence checks do students or researchers commonly perform before interpreting DNA data?*
2. *What do existing tools require users to enter, and how do they display their results?*
3. *Would combining basic validation, comparison, and visual reporting make the workflow easier for a beginner?*

For your research, start with the official resources for NCBI BLAST, Benchling, and FastQC.

My verdict: SeqAudit is a strong candidate if you want a project that feels like a real bioinformatics utility, demonstrates actual algorithms, and remains achievable within your course constraints.

The key is to make the comparison and quality-control workflow your central feature, rather than adding lots of unrelated DNA tools.

i like idea 3 better

I think RestrictionMap is an excellent choice if you want to build something that feels like a real molecular biology tool. It addresses a specific laboratory task, involves genuine biological concepts, and gives you plenty of opportunities to demonstrate technical programming skills.

Let's develop the idea into a project that is both useful and realistic for your CS1 final.

🧬 *Your Project: RestrictionMap*

An interactive DNA restriction-site analysis tool

1. The problem you're solving

What problem do scientists face?

Molecular biologists often use restriction enzymes to cut DNA at particular nucleotide sequences. Researchers working with plasmids or other DNA fragments may need to determine which restriction sites are present and what DNA fragments could result from cutting at those sites.

Doing this manually can become tedious when a sequence contains many nucleotides or when several enzymes need to be compared.

For example, a researcher might want to answer:

- *Does this DNA sequence contain a recognition site for a particular restriction enzyme?*
- *Where along the sequence are the sites located?*
- *How many times does each enzyme recognize the sequence?*
- *If the DNA is cut at these sites, what fragment lengths are expected?*
- *Which of several enzymes produces a particular pattern of fragments?*

Your application would automate these calculations and display the results in an easy-to-understand format.

Your core problem statement could be:

Researchers and biology students sometimes need to inspect DNA sequences to determine where restriction enzymes recognize and cut the DNA. RestrictionMap will automate restriction-site detection and predicted fragment-length calculations, helping users explore

DNA sequences without manually searching for recognition patterns.

This is a legitimate scientific computing problem, and you can implement its basic functionality using Python string processing.

---

2. What existing tools already do

Before deciding what makes your application useful, you should understand the existing tools.

Tool 1: EMBOSS Restrict

Official resource: EMBOSS Restrict documentation

EMBOSS Restrict identifies restriction enzyme recognition sites in DNA sequences. It can report their positions and information about the resulting fragments.

What this means for your project: You don't need to invent restriction-site detection. You can create a simpler interface that makes the results easier for a beginner to understand.

Tool 2: Benchling

Official website: Benchling Molecular Biology

Benchling provides a broader molecular biology environment, including sequence analysis and visualization features.

What this means for your project: Your application can focus on one workflow rather than trying to reproduce a complete research platform.

Tool 3: NEBcutter

Official website: NEBcutter

NEBcutter is a tool for analyzing DNA sequences and identifying restriction enzyme cleavage sites.

What this means for your project: RestrictionMap would be entering an established category of bioinformatics software. Your differentiator should be its focused educational design and clear explanation of the calculations, rather than claiming to have more comprehensive analysis capabilities.

Your potential niche

I would position RestrictionMap as:

A lightweight, educational restriction analysis tool that lets users explore enzyme recognition sites, compare predicted digests, and understand how DNA fragments are calculated.

That's a realistic project goal.

---

3. How the application would work

Imagine a user has a DNA sequence and wants to investigate how two different restriction enzymes would cut it.

They open RestrictionMap and follow this workflow:

Step 1: Enter DNA

The user pastes a sequence into a text area.

For example:

```
ATGGAATTCGCTGAATTCACG
```

Step 2: Select an enzyme

The application offers a small predefined collection of enzymes and their recognition sequences.

For example, EcoRI recognizes:

```
GAATTC
```

Step 3: Analyze the sequence

The application searches for occurrences of the recognition pattern and records their positions.

Step 4: Calculate fragment lengths

The application uses the positions of the cuts to calculate the lengths of the resulting DNA fragments.

Step 5: Display the results

The user sees:

- *The selected enzyme and its recognition sequence.*
- *The number of recognition sites.*
- *The positions of those sites.*
- *The predicted fragment lengths.*
- *A visual representation of the sequence and cut positions.*

Step 6: Compare enzymes

The user selects a second enzyme and compares the results.

This is the core application. Everything else should build on it.

---

4. The features I would build

I'd organize the project into three levels so that you can get a working version running before attempting the more advanced features.

Level 1: Essential features

These are the features I would consider mandatory for your own development plan.

A. DNA sequence input and validation

Users can paste a DNA sequence into a `TextArea`*.*

Your program:

- *Removes whitespace and line breaks.*
- *Converts letters to uppercase.*
- *Checks for invalid characters.*
- *Rejects empty input.*
- *Stores the sequence for analysis.*

You could also allow users to enter a sequence name.

B. Restriction enzyme selection

Create a small predefined collection of enzymes.

For example:

Enzyme	*Recognition sequence*
EcoRI	*GAATTC*
BamHI	*GGATCC*
HindIII	*AAGCTT*
HaeIII	*GGCC*

These are examples of commonly used restriction enzymes and their recognition

sequences.

You do not need hundreds of enzymes. Start with four to six well-understood examples.

C. Restriction-site detection

Search the DNA sequence for every occurrence of the selected recognition pattern.

For example, if the DNA sequence contains two occurrences of `GAATTC`, your program should report both.

For each occurrence, record its starting position.

Be consistent about whether positions start at zero or one. For a user-facing biology tool, one-based positions are generally easier to understand.

D. Predicted fragment lengths

For a simple linear DNA molecule, calculate the distances between consecutive cut positions, including the beginning and end of the sequence.

Suppose a hypothetical DNA molecule is 1,000 base pairs long and has cut positions at 200 and 700.

The predicted fragment lengths would be:

- *200 bp*
- *500 bp*
- *300 bp*

The lengths should add up to the original sequence length.

That gives you a useful correctness check for your code.

Important biological detail: Restriction recognition sites and cleavage positions are not always interchangeable. For a correct fragment calculation, your implementation must account for where each enzyme actually cuts relative to its recognition sequence. For the initial version, choose enzymes with well-defined, straightforward cleavage patterns and document your assumptions.

Level 2: Features that make it impressive

Once the essential features work, consider these additions.

E. Compare two enzymes

Let users analyze the same DNA sequence using two different enzymes.

Show a comparison table:

Measurement	*Enzyme A*	*Enzyme B*
Recognition sites	2	1
Predicted fragments	3	2

Smallest fragment	*150 bp*	*350 bp*
Largest fragment	*500 bp*	*650 bp*

The values above are illustrative rather than experimental results.

This feature involves useful data processing and gives your application a clear purpose beyond displaying a single result.

F. Visual restriction map

Display a horizontal representation of the DNA molecule, with marks showing the positions of restriction sites.

For example:

```
DNA sequence: 1,000 bp

0 bp        200 bp                  700 bp       1,000 bp
 |-------------|----------------------|-------------|
              EcoRI                  EcoRI
```

You can initially render a simple textual map using Drafter components. If time permits, you could use Matplotlib or another supported plotting approach to produce a more polished diagram.

G. Saved analyses

Let users save their analyses within the application and review previous results.

This feature naturally supports your required list of dataclass instances.

If you want results to persist between visits, investigate browser local storage. Otherwise, saved analyses can remain available for the current application session.

H. Downloadable report

Generate a text or CSV report containing:

- *The DNA sequence name.*
- *The selected enzyme.*
- *Recognition sequence.*
- *Recognition-site positions.*
- *Predicted fragment lengths.*

Drafter's supported download capabilities would determine the simplest implementation. You can leave this feature until after the main workflow works.

---

5. How to structure the Python program

Your rubric requires at least two dataclasses, including a collection of instances in your application state.

RestrictionMap fits this requirement naturally.

Dataclass 1: `DNASequence`

This represents the DNA molecule being analyzed.

Field	Type	Purpose
`name`	`str`	User-provided sequence name
`sequence`	`str`	DNA nucleotide sequence
`length`	`int`	Number of nucleotides
`description`	`str`	Optional notes about the sequence

Dataclass 2: `RestrictionAnalysis`

This represents the results of analyzing one sequence with one enzyme.

Field	Type	Purpose
`sequence_name`	`str`	Name of the analyzed DNA sequence
`enzyme_name`	`str`	Selected restriction enzyme
`recognition_sequence`	`str`	Nucleotide pattern recognized by the enzyme

`site_positions`	`list[int]`	Positions of detected recognition sites
`fragment_lengths`	`list[int]`	Calculated DNA fragment lengths

You can add fields later if needed, but these are enough to support the main workflow.

Your `State` dataclass

I would consider these fields:

Field	How it changes
`sequences`	Add new DNA sequences
`analyses`	Save new restriction analysis results
`selected_sequence`	Change the sequence being analyzed
`selected_enzyme`	Change the selected enzyme
`comparison_mode`	Enable or disable enzyme comparison

The first four can be meaningfully modified by different routes. That gives you room to meet the state requirements.

For example:

- *The sequence-input route adds to `sequences`.*
- *The analysis route adds to `analyses`.*
- *Selecting a sequence updates `selected_sequence`.*
- *Selecting an enzyme updates `selected_enzyme`.*

Make sure these are actual state changes, not just values that are initialized and never updated.

---

6. Your proposed website structure

You need at least seven routes and five pages. Here's a structure that naturally satisfies those requirements.

Route	*Purpose*
/	*Homepage and application overview*
/input	*Enter a DNA sequence*
/sequences	*View stored DNA sequences*

`/select-enzyme`	Select a restriction enzyme
`/analyze`	Run restriction-site analysis
`/results`	View recognition sites and fragment lengths
`/compare`	Compare two enzyme analyses
`/saved-analyses`	Review saved results
`/about`	Explain the biology and model assumptions

That's nine routes.

Some routes can share visual components, but I'd make sure at least five of them have clearly distinct page designs or purposes.

You can implement the main interface using Drafter components such as buttons, text boxes, text areas, select boxes, and checkboxes.

---

7. How this project meets your rubric

Requirement	Planned implementation
7+ routes	Nine routes
5+ pages	Homepage, Sequence Input, Sequence Library, Enzyme Selection, Results, Comparison, and Saved Analyses
`State` with at least 4 fields	Five meaningful state fields
Second dataclass	`DNASequence` and `RestrictionAnalysis`
Four modified state fields	Sequences, analyses, selected sequence, selected enzyme
Three input fields	Sequence name, DNA sequence, enzyme selection, plus optional comparison controls
Three meaningful `if` statements	Empty-sequence validation, invalid-character detection, and checking whether recognition sites exist
Meaningful loop	Iterate through sequences or recognition sites to display results and calculate fragment lengths

Legitimate purpose	*Analyze DNA restriction sites and predicted fragments*
No global variables	*Keep changing application data in `State`*
Drafter library	*Use Drafter routes and components*
Advanced techniques documented	*Explain any algorithms beyond course material in comments or docstrings*

You'll also need to write unit tests for your functions where possible and test your routes. You can test the underlying sequence-analysis functions independently of the interface, which should make debugging easier.

---

8. What could make RestrictionMap stand out?

If you want this to look like a genuine scientific application rather than a basic class assignment, I would prioritize three things.

1. Correctness

Make sure recognition sites are detected accurately and fragment lengths are calculated according to documented biological assumptions.

2. Clear visualizations

Give users a sequence map, a list of cut sites, and a fragment-length table. These outputs make the results easier to interpret.

3. Scientific explanations

For each analysis, explain what the output means and what the application does not establish.

For example, identifying a restriction site does not establish that an enzyme will necessarily cut the DNA under a particular laboratory protocol. Actual digestion depends on experimental conditions and other factors.

These explanations will also help you demonstrate that you understand the generated code and the biology behind it.

---

9. Your next step: Define the exact scope

Before you complete your project-pitch worksheet, I recommend making one decision.

Which version of RestrictionMap do you want to build?

A. Restriction Site Finder — Moderate difficulty

Users paste a DNA sequence, choose an enzyme, and see where its recognition sites occur. The application calculates predicted fragment lengths for a simple linear DNA

molecule.

B. RestrictionMap Lab — Challenging but manageable

Everything in A, plus a visual DNA map, comparison of two enzymes, and a saved-analysis history.

C. RestrictionMap Explorer — More ambitious

Everything in B, plus circular plasmid support, more detailed enzyme-cutting behavior, and downloadable reports.

My recommendation is B. It provides enough technical depth to stand out, naturally satisfies the assignment requirements, and remains realistic for a three-week project.

I'd build the sequence analysis and fragment calculations first, then add the visual map and comparison features. That way, you will have a working scientific application even if you run out of time for the more advanced features.

Final Project Roadmap

The final project will involve creating a web application with Drafter. The different steps of the project will be spread across the last several weeks of class.

- Week 10: Brainstorming & Ideation - You will come up with an idea for your web application. You will brainstorm its goal, details, and functionality.
- Week 11: Design - You will plan how the site will look and how you will implement the requirements listed on this page.
- Week 12: Development & Deployment - You will develop the code for your website

and deploy it.

- Week 13: User Testing - You will show your website to peers and get feedback on their experiences using it.
- Week 14: Updates based on Feedback - You will make updates based on the feedback you receive and reflect on the process of creating your web application.

Site Functionality Requirements

Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.
- At least 5 pages.
- The implementation of the *State* dataclass must have at least 4 fields.
    - At least one field must be a list of class instances or a dictionary of class instances.
        - This means that there must be a second dataclass.
    - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in . Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of)
- Unit tested your functions where possible, being : each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
    - If a function can't be unit tested, you must note as much in a docstring or comment.
- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good as the first line of your function's body
- Decomposed complicated functions with helper functions

Your TA will be grading your code's quality based on the ideas above.

Repository Requirements

Your final project's GitHub repository should meet the following requirements.

- Planning document should be uploaded to the repository.
- Website should be deployed without error.
- Site information should be set in `main.py` using `set_site_information(...)`.
- When changing/adding files in the repository, commit messages should be concise and descriptive, not the default "Update (filename)" message.
- A video fulfilling the following requirements should be linked in the repository.
    - Shows functioning deployed website
    - Audible and visible
    - Accesses and explains all necessary parts of the website
- All conversations with any AI should be uploaded to the repository.

The above are either present on the final rubric or are necessary for your site to be graded. help me come up with a project idea based on these intructions

…

I recommend choosing a project that solves a problem you actually experience as a college student. The best project for this assignment will be useful enough to have a legitimate purpose, creative enough to impress your TA, and simple enough to finish in three weeks using Python and Drafter.

Your website also needs at least 7 routes, 5 pages, two dataclasses, 4 meaningfully modified state fields, 3 input fields, and a meaningful loop. I'll suggest ideas that naturally satisfy those requirements rather than forcing you to add random features.

1. My Top 5 Project Ideas

Idea #1: Student Survival Hub 🎓

My top recommendation!

The problem

College students often struggle to keep track of assignments, exams, personal goals, and study sessions. Their information ends up scattered across notebooks, calendars, and different apps.

Your solution

Build a student dashboard where users can manage assignments, track their progress, and organize their study sessions in one place.

Features

- Dashboard: Show the number of assignments due soon, completed tasks, and upcoming exams.
- Assignment Tracker: Add assignments with a course name, due date, and priority.
- Study Planner: Create study sessions for different courses.
- Progress Tracker: Mark assignments as completed and calculate completion percentages.
- Course Manager: Organize assignments by course.
- Productivity Statistics: Display a graph of completed versus incomplete assignments.
- Study Tips: Display suggestions based on how many assignments remain incomplete.

Example of how it works

A student adds three assignments:

- CISC 106: Python Project — High Priority
- MATH 241: Homework 5 — Medium Priority
- ENGL 110: Essay — Low Priority

The dashboard displays the assignments and highlights the highest-priority task.

When the student marks the Python project complete, the dashboard updates its statistics.

How it satisfies the requirements

You can implement the project using two dataclasses.

Dataclass 1: Assignment

Field	Purpose
`title`	Assignment name
`course`	Course associated with it
`due_date`	Assignment deadline
`priority`	High, medium, or low
`completed`	Whether the assignment is finished

Dataclass 2: StudySession

Field	Purpose
`course`	Course being studied
`duration`	Study duration in minutes
`topic`	Topic covered

`completed`	Whether the session has been completed

Your `State` dataclass could contain these fields:

State field	How you modify it
`assignments`	Add assignments and update completion status
`study_sessions`	Add study sessions
`selected_course`	Update the course selected by the user
`study_goal`	Update the user's daily study goal
`current_page`	Track or update navigation state, if needed

You only need four meaningfully modified state fields, including the collection of dataclass instances. These five provide flexibility.

Example of a meaningful loop:

Loop through all assignments and display each one. Use an `if` statement to display a different message depending on whether an assignment is completed or overdue.

Possible routes and pages

Drafter routes are URL endpoints, so you can have more routes than distinct page designs.

Route	Page or function
`/`	Dashboard
`/assignments`	View all assignments
`/add-assignment`	Add an assignment
`/study-sessions`	View study sessions
`/add-session`	Add a study session
`/progress`	View productivity statistics
`/courses`	Filter assignments by course
`/complete-assignment`	Mark an assignment complete

This gives you 8 routes and 7 distinct page designs or views, depending on how you structure your application.

Difficulty

Beginner to intermediate.

Most of the functionality involves lists, dataclasses, loops, `if` statements, and input

fields—all appropriate for an introductory Python course.

You could also use browser local storage to preserve information between visits, if your course setup supports it. Without persistent storage, data may reset when the application reloads.

Why I recommend it: It's practical, easy to demonstrate, and naturally satisfies almost every programming requirement.

Idea #2: Budget Buddy 💸

A personal spending tracker for college students.

The problem

College students often have limited budgets and may not know where their money goes each month.

Your solution

Create a simple application that lets users record expenses, categorize purchases, and compare spending against a budget.

Features

- Set a monthly spending limit.
- Add expenses.
- Categorize expenses as food, transportation, entertainment, or school supplies.
- Display total spending.
- Calculate remaining budget.
- Show spending by category.
- Display a warning when spending exceeds a selected threshold.

Dataclasses

Expense

- `name`
- `amount`
- `category`
- `date`

BudgetCategory

- `name`
- `limit`
- `spent`

The `State` dataclass could track:

- `expenses`: a list of `Expense` objects.
- `categories`: a list of `BudgetCategory` objects.
- `monthly_budget`: the overall spending limit.
- `selected_category`: the currently selected category.
- `show_completed`: an optional display preference, if you give it a meaningful purpose.

Example

Suppose the user has a $300 monthly food budget and records:

- $12 for lunch.
- $45 for groceries.
- $18 for dinner.

The application calculates:

- Total spent: $75.
- Remaining budget: $225.

- Percentage spent: 25%.

It could also show a bar chart of spending by category.

Possible routes

1. Dashboard
2. Add expense
3. View expenses
4. View categories
5. Edit budget
6. Spending statistics
7. Filter expenses
8. Budget tips

Difficulty

Beginner to intermediate.

The calculations are straightforward, and the visualizations can make the finished project look impressive.

Important limitation: This should be a manual expense tracker, not a banking application. Connecting bank accounts, importing transactions automatically, or processing payments would add unnecessary complexity.

Idea #3: Campus Explorer 🗺️

A guide to interesting places around campus.

The problem

New students may not know where to study, eat, relax, or find useful campus resources.

Your solution

Build a campus guide that helps students find places based on their needs.

You could focus on the University of Delaware or design a guide for college campuses generally.

Features

- Browse campus locations.
- Filter locations by category.
- Save favorite places.
- Rate locations.
- Search for a place by name.
- Display location details.
- Recommend places based on a user's preferences.

Dataclasses

CampusLocation

- `name`
- `category`
- `description`
- `rating`
- `indoor`

Review

- `location_name`
- `rating`
- `comment`
- `visit_date`

Possible routes

1. Home
2. Browse locations
3. Search locations
4. Location details
5. Favorites
6. Submit review
7. View reviews
8. Recommendations

Difficulty

Beginner to intermediate, depending on how you implement the map.

The easiest version uses a predefined list of campus locations rather than an interactive map. You can display names, descriptions, and categories using ordinary Drafter components.

You could add a map later if time permits.

Important limitation: Don't make live building availability, real-time dining menus, or live campus navigation essential features. These could require external data sources or APIs that may not be available.

Idea #4: Meal Planner & Recipe Finder 🍳

A tool that helps students decide what to eat.

The problem

Students often have limited time and money to cook, and deciding what to make can be

difficult.

Your solution

Create a meal planner that recommends recipes based on available ingredients, budget, and dietary preferences.

Features

- Browse a collection of recipes.
- Search by recipe name.
- Filter by meal type.
- Select available ingredients.
- Calculate estimated meal costs.
- Generate a grocery list.
- Save favorite recipes.

Dataclasses

Recipe

- `name`
- `ingredients`
- `instructions`
- `estimated_cost`

GroceryItem

- `name`
- `quantity`
- `purchased`

Possible routes

1. Home

2. Browse recipes
3. Search recipes
4. Recipe details
5. Meal planner
6. Grocery list
7. Favorite recipes
8. Budget-friendly recipes

Difficulty

Beginner to intermediate.

You can create a small collection of sample recipes directly in your application. A large external recipe database or an AI-powered recipe generator is unnecessary.

The most interesting programming component would be comparing the ingredients the user has with the ingredients each recipe requires.

For example, if a user selects eggs, rice, and cheese, the application can identify recipes that use those ingredients and show how closely each recipe matches.

Idea #5: Habit Tracker & Personal Growth Dashboard 🌱

A visual tool for building better habits.

The problem

People often struggle to maintain habits because they forget to track their progress or find it difficult to see improvement over time.

Your solution

Build an application where users define habits, record daily progress, and view their

consistency.

Features

- Create new habits.
- Set daily or weekly goals.
- Mark habits as completed.
- Track streaks.
- View completion percentages.
- Display progress charts.
- Receive encouraging messages based on progress.

Dataclasses

Habit

- `name`
- `category`
- `goal`
- `completed_count`

HabitEntry

- `habit_name`
- `date`
- `completed`

Possible routes

1. Dashboard
2. Add habit
3. View habits
4. Record daily progress
5. Progress statistics

6. Habit details
7. Completed habits
8. Weekly summary

Difficulty

Beginner.

This is one of the simplest projects to implement correctly. The challenge is making it visually appealing and ensuring that tracking progress is meaningful.

You could add a calendar or a graph showing completion rates over time.

---

2. Which Project Is Best for Your Assignment?

Here's how I would compare the five ideas.

Criteria	Student Survival Hub	Budget Buddy	Campus Explorer	Meal Planner	Habit Tracker
Legitimate purpose	Excellent	Excellent	Excellent	Excellent	Excellent
Easy to satisfy requirements	Excellent	Excellent	Very good	Very good	Excellent
Interesting user interface	Excellent	Excellent	Excellent	Excellent	Very good

Simple Python logic	Very good	Very good	Very good	Good	Excellent
Opportunities for charts	Excellent	Excellent	Optional	Optional	Excellent
Potential usefulness after class	Excellent	Excellent	Very good	Excellent	Very good
Overall difficulty	Moderate	Moderate	Moderate	Moderate	Easy

My ranking:

1. Student Survival Hub — best overall balance of usefulness, functionality, and manageable complexity.
2. Budget Buddy — best if you enjoy working with numbers, calculations, and graphs.
3. Campus Explorer — best if you want a visually interesting project with a campus theme.
4. Meal Planner — best if you enjoy food, searching, and filtering.
5. Habit Tracker — best if your priority is finishing a reliable application without excessive complexity.

All five can meet the rubric. The quality of the implementation matters more than the idea alone.

---

3. My Recommendation: Build the Student Survival Hub

If I were planning this project for a CS1 student, I would choose the Student Survival Hub and make its central purpose:

Help college students organize their assignments, plan study sessions, and monitor their academic progress in one place.

The key is to build a small but complete application instead of attempting to recreate an entire productivity platform.

Your proposed five main pages

Page 1: Dashboard

The first page a user sees.

Display:

- Total assignments.
- Number completed.
- Number remaining.
- Assignments due soon.
- Today's study goal.
- A list of priority assignments.

Page 2: Assignment Manager

Let users:

- View all assignments.
- Add an assignment.
- Select its priority.
- Enter a due date.
- Mark it as completed.

Page 3: Study Planner

Let users:

- Add a study session.
- Choose a course.
- Enter the topic.
- Enter the planned duration.
- Mark the session as completed.

Page 4: Progress

Display:

- Completed versus incomplete assignments.
- Total study time.
- Study time by course.
- Completion percentages.

You could use a simple Matplotlib chart if it is supported in your Drafter environment.

Page 5: Course Explorer

Let users select a course and view:

- Its assignments.
- Upcoming deadlines.
- Completed work.
- Associated study sessions.

These pages would provide the core experience. Additional routes can handle actions such as adding assignments, completing tasks, and filtering records.

---

4. How to Meet Every Programming Requirement

This is the most important planning step. You want to ensure that your idea naturally supports the rubric.

Requirement	How your application will meet it
At least 7 routes	Create routes for the dashboard, assignment list, assignment form, study sessions, session form, progress, courses, and task completion.
At least 5 pages	Build five main views: Dashboard, Assignments, Study Planner, Progress, and Courses.
`State` dataclass with 4+ fields	Track assignments, study sessions, selected course, study goal, and other useful settings.
Second dataclass	Create `Assignment` and `StudySession`.
At least 4 modified state fields	Add assignments, add sessions, change assignment completion status, and update the study goal.
At least 3 input fields	Use a `TextBox` for a title, a `SelectBox` for priority, and a `CheckBox` for completion, along with other useful inputs.
At least 3 meaningful `if` statements	Check for empty titles, determine whether tasks are completed, and identify assignments due soon.

At least 1 meaningful loop	Iterate through the assignment objects to display them, filter them, or calculate completion statistics.
Legitimate purpose	Help students organize their academic responsibilities.
No global variables	Store changing application data in `State`; use global constants only where appropriate.
Use Drafter	Build the website using Drafter routes and components.
Document advanced techniques	Explain any techniques beyond the course material in comments or docstrings.

One critical detail about state

Don't just declare five fields in your `State` dataclass and leave them unchanged.

The rubric specifically requires meaningful modifications.

For example:

- Adding an assignment changes `state.assignments`.
- Adding a study session changes `state.study_sessions`.
- Selecting a different course changes `state.selected_course`.
- Updating the daily goal changes `state.study_goal`.

Also, when a user marks an assignment complete, update the relevant `Assignment` object within the list. Simply displaying a checkbox without changing the underlying data would not

be sufficient.

Before coding, verify that each route actually modifies the state it is supposed to modify.

---

5. Keep the Project Realistic for Drafter

Because Drafter runs Python in the browser using Pyodide, you can accomplish quite a lot without a traditional backend. However, you should plan around its limitations.

Features you can reasonably build

- Adding and displaying assignments.
- Filtering assignments by course or priority.
- Calculating completion percentages.
- Tracking study-session durations.
- Displaying simple charts.
- Saving data using browser local storage, if supported by your setup.
- Building multiple pages and routes.
- Creating a consistent interface with navigation buttons.

Features I would avoid initially

1. User accounts and passwords

A proper login system usually requires a backend and secure authentication. It isn't necessary for this project.

2. Shared assignments across different students

Your application doesn't have a backend database, so you shouldn't assume that different students can share a live assignment list.

3. Automatic Canvas assignment imports

This could require API access, authentication, and integration work beyond the scope of a three-week introductory project.

4. AI-generated study schedules

An advanced AI integration adds unnecessary complexity. A simple scheduling algorithm based on due dates and priorities is easier to build, explain, and test.

5. Complicated calendar integrations

Start by accepting dates as input and displaying assignments in a list. You can add a calendar-style view later if the core application works.

A note about saving data

You can investigate using browser local storage to keep assignments between visits. However, don't assume that ordinary Python variables will persist when the page reloads.

If you use local storage, test it early. If you cannot get it working, the application can still be useful during a session, but you should clearly understand that limitation.

---

6. Your Three-Week Development Plan

Your project roadmap already provides a sensible sequence. Here's how I would approach it.

Week 10: Brainstorming and Ideation

Goal: Decide what the application will do.

1. Define the problem: students struggle to organize academic responsibilities.
2. Choose your primary users: college students.

3. Identify your main features.
4. Research existing solutions.
5. Define your two dataclasses.
6. Plan your `State` fields.
7. Identify your routes.
8. Complete the project pitch worksheet.

Don't start by trying to implement every possible feature. Decide which features are essential.

Week 11: Design

Goal: Plan the application before writing most of the code.

1. Sketch the five main pages.
2. Decide how users navigate between pages.
3. Define the fields for each dataclass.
4. Plan each route and its purpose.
5. Decide which inputs appear on each form.
6. Plan how assignment filtering and completion will work.
7. Identify how each rubric requirement will be satisfied.

At the end of this week, you should know what each page does and what data it needs.

Week 12: Development and Deployment

Goal: Build a working application.

Implement features in this order:

1. Create the `State`, `Assignment`, and `StudySession` dataclasses.
2. Build the dashboard and navigation.
3. Add the assignment form and assignment list.
4. Implement assignment completion.

5. Add study sessions.
6. Add filtering and progress calculations.
7. Build the remaining pages.
8. Add charts or other visual enhancements.
9. Test and deploy the application.

Get a working version deployed before adding advanced features. A simple working application is much better than an ambitious one that fails to run.

Week 13: User Testing

Goal: Find out whether people can actually use your application.

Ask classmates to complete tasks such as:

- Add an assignment.
- Find assignments for a particular course.
- Mark an assignment complete.
- Add a study session.
- Find their progress statistics.

Ask them:

- Was anything confusing?
- Could you find the features you needed?
- Did any buttons behave unexpectedly?
- What is one improvement you would suggest?

Record their feedback instead of relying on memory.

Week 14: Updates Based on Feedback

Goal: Improve the application and demonstrate that it works reliably.

1. Choose the most important feedback.

2. Fix bugs and confusing navigation.
3. Improve the interface.
4. Retest the affected features.
5. Verify that every route works.
6. Check that your code satisfies the rubric.
7. Record a video demonstrating the deployed website.
8. Upload your planning document and AI conversations to the repository.

---

7. What Existing Applications Could You Research?

For the Student Survival Hub, useful prior solutions to investigate include:

- Notion — Notion official website
- Todoist — Todoist official website
- Google Calendar — Google Calendar official website

These tools already help people manage tasks, schedules, and personal information.

Your project doesn't need to outperform them. Instead, identify a specific problem your simplified application addresses.

For example, you could focus on showing students which assignments deserve attention first by combining due dates, priority, and completion status.

That gives your application a clear purpose beyond simply storing tasks.

---

8. Let's Narrow It Down to Your Interests

The Student Survival Hub is my strongest recommendation, but the best project is one you will enjoy building.

Answer these three questions, and I can help you settle on an idea that fits your interests

and the rubric.

1. Which category interests you most?

- A. College life and productivity
- B. Money, budgeting, and spending
- C. Food, cooking, and meal planning
- D. Fitness, habits, and personal improvement
- E. Games, entertainment, and interactive experiences
- F. Campus life and finding interesting places
- G. Something else

2. What is your main goal?

- A. Make something genuinely useful.
- B. Make something visually impressive.
- C. Keep the coding relatively easy.
- D. Build something creative and unique.
- E. Balance all four.

3. How comfortable are you with Python right now?

- A. Basic Python: variables, `if` statements, loops, and functions.
- B. Comfortable with lists, dictionaries, and dataclasses.
- C. Comfortable with most of the course material and ready for a challenge.

Once you've answered, I can help you choose a project and map it directly to the assignment requirements, including the two dataclasses, state fields, eight routes, and the features that will make it stand out.

i want something more technical, i am interested in biology, give me ideas related to that

If you're interested in biology and want something more technical, I'd lean toward building a scientific web application rather than a standard productivity app. You could make

something that models biological systems, analyzes data, visualizes scientific patterns, or simulates experiments.

Since you're using Python with Drafter and Pyodide, you have some interesting options: you can use libraries like NumPy, pandas, and Matplotlib (if supported in your environment) to perform calculations and visualize data directly in the browser.

Here are 7 biology-related project ideas, ranked with your assignment requirements and your interest in technical complexity in mind.

1. 🧬 BioSim: Biological Systems Simulator

My top recommendation if you want something technical, impressive, and manageable.

The concept

Build an interactive simulator that models how biological populations change over time.

For example, you could simulate:

- Bacteria growing in a culture.
- A predator-prey ecosystem.
- Competition between two species.
- The spread of a non-medical, fictional infection through a simulated population.

The user changes parameters, runs a simulation, and sees how the system behaves.

Example: Predator-prey simulator

Imagine modeling rabbits and foxes.

The user enters:

- Initial rabbit population.
- Initial fox population.
- Rabbit reproduction rate.
- Fox predation rate.
- Simulation duration.

Your program calculates how both populations change over time and plots the results.

For a more mathematical approach, you could use the classic Lotka–Volterra equations:

$$\frac{dR}{dt} = \alpha R - \beta RF$$

$$\frac{dR}{dt} = \alpha R - \beta RF$$

$$\frac{dF}{dt} = \delta RF - \gamma F$$

$$\frac{dF}{dt} = \delta RF - \gamma F$$

Where:

- R
- R = rabbit population.
- F
- F = fox population.
- α
- α = rabbit growth rate.
- β
- β = predation rate.
- δ
- δ = fox population growth from prey.
- γ
- γ = fox death rate.

You don't need to understand advanced differential equations to build a simplified version. You could approximate the population changes in discrete time steps using basic Python arithmetic and loops.

Pages and routes

Route	Purpose
`/`	Introduction and simulator overview
`/simulator`	Enter parameters and run simulations

`/results`	View the latest simulation results
`/population-graph`	Visualize population changes
`/compare`	Compare two simulations
`/saved-simulations`	View saved simulations
`/species`	Learn about the modeled species
`/about-model`	Explain the equations and assumptions

How it satisfies your rubric

Dataclass 1: `Simulation`

- `name`
- `rabbit_start`
- `fox_start`
- `duration`
- `growth_rate`

Dataclass 2: `PopulationPoint`

- `time_step`
- `rabbit_population`
- `fox_population`
- `total_population`

Your `State` could contain:

- `simulations`: a list of `Simulation` objects.
- `population_history`: a list of `PopulationPoint` objects.
- `selected_simulation`: the simulation currently being viewed.
- `simulation_count`: the number of simulations run.
- `time_steps`: the number of steps in the current simulation.

You could modify the first four fields through different routes, ensuring they genuinely change during program execution.

A meaningful loop would iterate through the simulation's population history to display values or calculate maximum and minimum populations.

What makes it impressive?

You could let users compare different scenarios, such as:

- What happens when rabbits reproduce twice as quickly?
- What happens when the initial fox population increases?
- Under what conditions does one population collapse?

A well-designed scientific graph and a clear explanation of the model's assumptions would make this feel like a real scientific tool.

Difficulty: 4/5. The mathematics and plotting require more thought than a standard CRUD application, but the model can be implemented using loops, lists, arithmetic, and functions.

Important: The results are predictions from a simplified mathematical model, not accurate forecasts of real ecosystems.

---

2. 🧬 DNA Lab: DNA Sequence Analyzer

Best if you are interested in genetics, molecular biology, or bioinformatics.

The concept

Build a mini bioinformatics platform that analyzes DNA sequences.

Users can paste a DNA sequence, and your application analyzes its composition and identifies interesting patterns.

For example, the input might be:

```
ATGCGCATATGCCGTA
```

Your program could calculate:

- Sequence length.
- Number of adenine, thymine, cytosine, and guanine nucleotides.
- GC content percentage.
- Reverse complement.
- Potential start and stop codons.

- Positions of selected nucleotide patterns.

Features

1. Sequence analysis

Count each nucleotide and calculate the GC percentage:

$$GC\%=G+CA+T+G+C\times100$$

$$GC\%=$$

$$A+T+G+C$$

$$G+C$$

$$\times100$$

2. Reverse complement

For a DNA sequence, replace:

- A with T.
- T with A.
- C with G.
- G with C.

Then reverse the resulting sequence.

3. Codon analysis

Divide a sequence into groups of three nucleotides and identify codons such as ATG or the standard DNA stop codons TAA, TAG, and TGA.

4. Sequence comparison

Compare two sequences and calculate the percentage of matching positions. For simplicity, you could require the sequences to have equal lengths.

5. Pattern search

Allow the user to enter a nucleotide pattern and find its positions in the sequence.

6. Visualizations

Use a bar chart to display nucleotide counts and a second chart to compare the composition of two sequences.

Pages and routes

Route	Purpose
/	DNA Lab homepage
`/analyze`	Analyze a DNA sequence

`/composition`	Display nucleotide statistics
`/reverse-complement`	Generate a reverse complement
`/codons`	Explore codons
`/compare`	Compare two sequences
`/pattern-search`	Find a nucleotide pattern
`/history`	View previous analyses

Dataclasses

`DNASequence`

- `name`
- `sequence`
- `length`
- `gc_content`

`AnalysisResult`

- `sequence_name`

- `a_count`
- `t_count`
- `c_count`
- `g_count`

Your `State` could hold:

- `sequences`: a list of `DNASequence` objects.
- `analysis_results`: a list of `AnalysisResult` objects.
- `current_sequence`: the sequence being analyzed.
- `comparison_sequence`: the second sequence.
- `search_pattern`: the pattern to find.

The first two fields can be modified as sequences are analyzed and results are stored. The other fields can change as users enter new inputs.

What makes it impressive?

You could make the application feel like a real bioinformatics tool, with a sequence input area, a nucleotide composition graph, color-coded DNA letters, and a results dashboard.

You could also include a built-in sample sequence so users can experiment immediately.

Difficulty: 3/5. This is very achievable with introductory Python, but the biological context makes it feel technical. Most operations involve string manipulation, loops, conditions, and arithmetic.

One scientific limitation: Finding a start codon or calculating GC content does not establish that a sequence is a functional gene. Your application should describe these as basic

sequence analyses, not definitive biological interpretations.

---

3. 🦠 MicrobeLab: Bacterial Growth Simulator

Best if you like microbiology and experimental science.

The concept

Build a simulator that models how a bacterial population grows over time under different environmental conditions.

The user chooses model parameters and sees how the population changes.

Features

- Set an initial bacterial population.
- Adjust the growth rate.
- Set a carrying capacity.
- Simulate growth over a selected time period.
- Compare two simulated conditions.
- Plot population size against time.
- Display doubling time under exponential growth assumptions.
- Explain the lag, exponential, stationary, and death phases of bacterial growth.

Example

Suppose you start with a hypothetical population of 100 bacteria.

Under an idealized exponential growth model:

$$N(t)=N_0e^{rt}$$

$N(t)=N$

0

e

rt

Where:

- $N(t)$
- $N(t)$ is the population at time
- t
- t.
- $N0$
- N
- 0
-
- is the initial population.
- r
- r is the modeled growth rate.

Your program could compare a low-growth-rate scenario with a high-growth-rate scenario and display both curves.

You could eventually introduce a logistic growth model that accounts for limited resources.

Potential routes

1. Home
2. New experiment
3. Growth simulation
4. Growth graph
5. Compare experiments
6. Experiment history
7. Growth calculator
8. Model explanation

Dataclasses

`GrowthExperiment`

- `name`
- `initial_population`
- `growth_rate`
- `duration`

`GrowthMeasurement`

- `time`
- `population`
- `experiment_name`

A loop can calculate the population at each time step and store the results in a list.

What makes it impressive?

The user can explore how changing one parameter affects an entire population trajectory.

You could display multiple curves on the same graph.

This is essentially a small computational biology application.

Difficulty: 3.5/5. The mathematics is manageable, although you'll need to decide how to handle very large population values and validate user inputs.

Important: Present this as a mathematical simulation. Avoid implying that the model predicts the behavior of a specific real microorganism without appropriate experimental data and assumptions.

---

4. 🧠 NeuroLab: Neuron Signal Simulator

Best if you like neuroscience, physiology, or electrical signals in biological systems.

The concept

Build an interactive educational simulator that demonstrates how a simplified neuron responds to stimulation.

A neuron communicates using changes in its membrane potential. Your application could show a simplified action potential waveform and let the user change the stimulus parameters.

Features

- Adjust stimulus strength.
- Adjust the firing threshold.
- Change the simulated time interval.
- Display a membrane-potential graph.

- Identify depolarization and repolarization.
- Compare responses under different parameter settings.
- Save simulations for comparison.
- Display educational explanations of the different phases.

How it works

You could implement a simplified threshold-based model:

1. Generate a series of time steps.
2. Calculate the simulated membrane potential.
3. Check whether the potential crosses the chosen threshold.
4. Generate a simplified spike when the threshold is reached.
5. Plot the results.

This would be an educational approximation rather than a complete biophysical model of a neuron.

Possible routes

1. Home
2. Neuron simulator
3. Stimulus settings
4. Signal graph
5. Compare simulations
6. Saved experiments
7. Neuron anatomy
8. Model explanation

Dataclasses

`NeuronSimulation`

- `name`
- `stimulus_strength`
- `threshold`
- `duration`

`SignalPoint`

- `time`
- `membrane_potential`
- `phase`

What makes it impressive?

The application could visually resemble a neuroscience lab tool, with adjustable parameters, interactive controls, and signal plots.

It offers a stronger technical identity than a standard data-entry application.

Difficulty: 4/5. Generating and plotting the waveform is manageable, but choosing a biologically reasonable model and explaining its limitations requires care.

---

5. 🌿 EcoSim: Ecosystem and Biodiversity Analyzer

Best if you like ecology, evolution, and environmental science.

The concept

Build an application that analyzes a hypothetical ecosystem containing several species.

Users can explore how species populations change and how biodiversity differs between environments.

Features

- Create a simulated ecosystem.
- Add species and their population sizes.
- Adjust reproduction or mortality rates.
- Track populations over time.
- Compare two ecosystems.
- Calculate species richness.
- Display population distributions.
- Visualize how biodiversity changes.

You could start with simple population models and later add interactions between species.

Dataclasses

`Species`

- `name`
- `population`
- `growth_rate`
- `environment`

`EcosystemSnapshot`

- `time_step`
- `species_populations`
- `species_count`

Possible routes

1. Home
2. Create ecosystem
3. Add species
4. Current populations
5. Simulate ecosystem
6. Compare ecosystems
7. Biodiversity statistics
8. Model explanation

What makes it impressive?

You can build an application that looks like a scientific ecology dashboard, with population charts and biodiversity statistics.

You could also demonstrate how removing one species affects a simplified food web, although modeling these interactions accurately adds complexity.

Difficulty: 4/5. The basic statistics are easy; modeling interactions among species is the challenging part.

---

6. 🧪 LabBench: Scientific Experiment Analyzer

Best if you want something that resembles a real research tool.

The concept

Create a general-purpose scientific data analysis application for biology experiments.

A researcher or student enters experimental measurements, and the program calculates summary statistics and visualizes the results.

For example, you could use simulated data from an experiment examining plant growth under different light conditions.

Features

- Create an experiment.
- Add observations.
- Group measurements by treatment.
- Calculate averages and ranges.
- Compare experimental groups.
- Plot results.
- Identify the maximum and minimum observations.
- Save experiments and view previous results.

Example

Suppose you have three hypothetical plant-growth groups:

Group	Heights (cm)
Low light	4, 5, 6, 4, 5

Medium light	7, 8, 9, 7, 8
High light	6, 7, 5, 8, 6

Your application calculates the average for each group and creates a bar chart.

This lets users explore whether the groups appear to differ.

Dataclasses

`Experiment`

- `name`
- `organism`
- `treatment`
- `description`

`Measurement`

- `experiment_name`
- `sample_id`
- `value`
- `unit`

Possible routes

1. Home
2. Create experiment

3. Add measurement
4. View measurements
5. Summary statistics
6. Compare groups
7. Data visualization
8. Experiment history

What makes it impressive?

This project is less about simulating biology and more about building a scientific computing application.

You could add CSV upload support, which would let users analyze data rather than manually entering every measurement. Start with manual input, however, so that file processing doesn't become a blocker.

Difficulty: 3.5/5. The statistics and graphs are straightforward, but organizing experimental data carefully takes some planning.

Scientific limitation: Differences between group averages alone do not prove that a treatment caused an effect. You should present descriptive statistics as summaries rather than definitive conclusions.

---

7. 🧬 EvolutionLab: Natural Selection Simulator

Best if you like evolutionary biology and computational modeling.

The concept

Create a simulated population of organisms with different inherited traits and model how the trait distribution changes over generations.

For example, imagine a hypothetical beetle population containing green and brown beetles.

In a green environment, the model assigns green beetles a higher survival probability. The user runs the simulation for several generations and observes changes in the population.

Features

- Set the initial population.
- Select the frequency of different traits.
- Adjust survival probabilities.
- Set the number of generations.
- Simulate reproduction and selection.
- Plot trait frequencies over time.
- Compare two environments.
- Explain natural selection.

Dataclasses

`Organism`

- `trait`
- `generation`
- `survival_probability`

`GenerationSummary`

- `generation_number`

- `green_count`
- `brown_count`
- `total_population`

Possible routes

1. Home
2. Configure population
3. Run evolution
4. Generation results
5. Trait-frequency graph
6. Compare environments
7. Saved simulations
8. Evolution concepts

What makes it impressive?

You could demonstrate an evolutionary process over multiple generations rather than simply displaying facts about evolution.

The application could include a graph showing the percentage of each trait over time.

Difficulty: 4/5. The model can be relatively simple, but random sampling, population updates, and explaining the assumptions require care.

You should make clear that the outcome is one possible simulation, not a universal prediction of evolution in real populations.

---

8. Comparison: Which Is the Best Fit?

Project	Main technical skill	Biology area	Difficulty	Visual potential
BioSim	Mathematical modeling	Ecology	4/5	Excellent
DNA Lab	String processing and algorithms	Genetics / bioinformatics	3/5	Excellent
MicrobeLab	Growth modeling	Microbiology	3.5/5	Excellent
NeuroLab	Signal generation and modeling	Neuroscience	4/5	Excellent
EcoSim	Population modeling and statistics	Ecology	4/5	Excellent
LabBench	Data analysis and statistics	Experimental biology	3.5/5	Excellent
EvolutionLab	Simulation and probability	Evolutionary biology	4/5	Excellent

These difficulty ratings are estimates, assuming you're comfortable with basic Python and have access to the relevant libraries.

My personal top three for you

🥇 First: DNA Lab

Choose this if you want a technical project with a clear biological purpose, interesting algorithms, and manageable complexity.

Why I like it:

- It uses real concepts from bioinformatics.
- The calculations are deterministic and easy to test.
- It naturally supports several pages and routes.
- It requires meaningful string processing, loops, and conditionals.
- It can have a professional scientific interface.
- You can add more advanced features without making the core project too difficult.

🥈 Second: BioSim

Choose this if you want the project to feel more like computational biology.

Why I like it:

- You can explore biological behavior by changing parameters.
- The math and graphs make the application feel technical.
- Comparing simulations gives you an interesting feature that goes beyond simple data entry.

- It demonstrates computational thinking and scientific modeling.

Its main drawback is that you need to understand and explain the model's equations and assumptions.

🥉 Third: NeuroLab

Choose this if neuroscience interests you most.

Why I like it:

- It has an interesting scientific subject.
- It provides opportunities for adjustable controls and dynamic visualizations.
- The results are easy to demonstrate in a video.
- It could look like a miniature neuroscience research tool.

Its main drawback is that a biologically credible model requires more care than a simple DNA analyzer.

---

9. An Idea That Combines the Best of Several Projects

If you want something particularly ambitious but still manageable, you could create BioLab: An Interactive Biology Computing Toolkit.

Instead of trying to build one enormous simulator, you could build a unified scientific application with three modules:

1. Sequence Analyzer: Analyze DNA composition and search for nucleotide patterns.
2. Population Simulator: Simulate biological population changes over time.
3. Experiment Analyzer: Calculate statistics and plot experimental measurements.

Each module would have its own pages, inputs, and results.

This would be a compelling demonstration of scientific programming, but it also has a risk: three separate modules mean more code, more testing, and more opportunities for bugs.

For a three-week CS1 project, I would only choose this approach if you keep each module very small and finish one before beginning another.

My recommendation is to choose one biological problem and solve it well. A polished DNA analyzer or population simulator is likely to be stronger than three unfinished tools.

Let's narrow it down

I have three questions that will help me identify the strongest project for you:

1. Which area of biology interests you most?

- A. Genetics, DNA, and molecular biology
- B. Neuroscience and the brain
- C. Microbiology, bacteria, and viruses
- D. Ecology, ecosystems, and evolution
- E. Human biology, physiology, and health
- F. Biology in general; I like the computational side most

2. What kind of technical challenge sounds most interesting?

- A. Algorithms that analyze biological sequences or data.
- B. Mathematical models and simulations.

- C. Graphs, statistics, and scientific data analysis.
- D. A combination of all three.

3. How ambitious do you want to be?

- A. Moderate: polished, technical, and relatively safe to finish.
- B. Challenging: something that stands out from typical CS1 projects.
- C. Very ambitious: willing to take on more complex biology and programming.

Your answers will help me narrow this down to one specific project concept with a clear purpose, technical features, and a realistic implementation plan that satisfies every major rubric requirement.

for the DNA lab, i want to be adressing a specific problem that is faced by scientists that my tools can solve, what are other tools doing and what are some applications i can focus on

DNA Lab: Turning a Bioinformatics Tool into a Solution for Scientists

I think this is a much stronger direction for your project. Instead of building a generic DNA analyzer that calculates GC content and generates reverse complements, you can identify a specific problem that researchers encounter when working with DNA sequences and build a lightweight tool that addresses it.

The key is to choose a problem that:

1. Scientists genuinely encounter.
2. Existing software already attempts to solve, so you can research how your tool could be different.
3. You can meaningfully address with Python, without needing a biological database or a powerful backend.

4. Allows you to demonstrate algorithms, data structures, visualization, and scientific reasoning.
5. Can be implemented and tested within your three-week development period.

One important distinction: you don't need to invent an entirely new bioinformatics problem. You can build a simpler, more focused solution for a particular type of user or workflow.

Here are the directions I would investigate first.

1. What problems do scientists actually face?

There are several different stages in DNA research, and each creates different computational problems.

Consider a simplified workflow for a molecular biology researcher:

1. Obtain a DNA sequence. A sequencing experiment produces data that needs to be processed.
2. Check the sequence. Researchers need to identify potential quality problems, unexpected characters, and other issues.
3. Compare sequences. They may want to determine how an experimental sequence differs from an expected reference.
4. Interpret the sequence. They may look for genes, coding regions, or particular DNA motifs.
5. Record and communicate results. They need to document findings so they can be reviewed and reproduced.

Each step presents an opportunity for a tool. However, existing software already handles many of these tasks, so your project needs a clearly defined scope.

Here are the main tools worth researching and what they tell us about potential project opportunities.

2. What existing bioinformatics tools do

A. NCBI BLAST — Finding similar DNA sequences

Official website: NCBI BLAST

The problem it solves: A researcher has a DNA sequence but wants to find similar sequences in a biological database.

For example, a researcher might have a sequence from an environmental sample and want to investigate which known organisms have similar sequences.

BLAST searches biological sequence databases and identifies regions of similarity. It is widely used for sequence identification and investigating evolutionary or functional relationships.

What your tool could learn from it:

- How to compare sequences.
- How to present matching regions.
- How to summarize similarity using understandable statistics.

What I would not attempt: Rebuilding BLAST. Its database searching, alignment heuristics, and statistical significance calculations are far beyond what you need for this project.

Instead, you could compare a small set of sequences provided by the user, without

searching an external database.

B. Benchling — Managing and analyzing DNA designs

Official website: Benchling Molecular Biology

The problem it solves: Molecular biologists need to design, inspect, analyze, and manage DNA sequences as part of research workflows.

Benchling offers features for sequence visualization, annotations, alignments, primer design, and other molecular biology tasks.

What your tool could learn from it:

- Organizing sequence information.
- Showing annotations and sequence features.
- Making analysis results easier to understand visually.
- Keeping a record of analyses.

The opportunity: Rather than building an entire molecular biology platform, focus on one narrow workflow that you can implement particularly clearly.

For example, your application could take a reference sequence and an experimental sequence and produce an easy-to-read discrepancy report.

That would be a small, focused application rather than a replacement for Benchling.

C. NCBI ORFfinder — Identifying potential coding regions

Official website: NCBI ORFfinder

The problem it solves: Researchers sometimes need to identify regions of a DNA sequence that could encode proteins.

ORFfinder identifies open reading frames (ORFs), reports their coordinates, and can provide their translated protein sequences.

What your tool could learn from it:

- Searching for patterns in DNA.
- Identifying start and stop codons.
- Calculating positions within a sequence.
- Presenting potential coding regions.

The opportunity: You could make an educational ORF exploration tool that shows how the three possible reading frames on each strand affect the results.

However, finding ORFs is an established task, so this idea would be most compelling if you had a specific audience or a distinct visualization approach.

D. FastQC — Assessing sequencing-read quality

Official website: FastQC project

The problem it solves: Researchers working with high-throughput sequencing data need to assess whether their sequencing reads exhibit potential quality problems before

downstream analysis.

FastQC generates quality-control reports for sequencing data. Tools in this workflow can help identify issues such as adapter contamination and low-quality reads.

What your tool could learn from it:

- Checking many sequences systematically.
- Flagging potential problems rather than simply displaying raw data.
- Generating a report that prioritizes issues for review.
- Visualizing statistics across a collection of sequences.

The opportunity: Build a lightweight sequence-quality screening tool for a small collection of DNA sequences.

One important distinction: FastQC analyzes sequencing-read data, including per-base quality information. If your input contains only plain DNA letters, you cannot reproduce those quality measurements. Your application would instead assess properties such as sequence length, nucleotide composition, ambiguous bases, and differences from an expected reference.

This is a more realistic scope for a browser-based CS1 project.

---

3. Five specific problems you could focus on

Now let's turn these observations into project ideas with concrete scientific purposes.

Idea 1: SequenceCheck — Finding unexpected differences in DNA

sequences

My favorite option for a technically focused project.

The scientific problem

Researchers may need to check whether an experimentally obtained DNA sequence matches an expected reference sequence.

For example, suppose a researcher expects a particular DNA fragment but obtains this sequence after an experiment:

Position	Expected sequence	Observed sequence
1–5	ATGCA	ATGCA
6–10	GTTAC	GTCAC
11–15	CAGTA	CAGTA

The sequences differ at one position: the seventh nucleotide.

Finding such differences by manually inspecting long sequences can be tedious and error-prone.

Your solution

Build SequenceCheck, a tool that compares an expected DNA sequence against an observed sequence and generates a discrepancy report.

The user enters the reference sequence and the observed sequence.

Your application:

1. Validates both sequences.
2. Checks whether they have the same length.
3. Compares corresponding positions.
4. Identifies mismatches.
5. Calculates the number and percentage of matching positions.
6. Displays a report showing where the sequences differ.

For example:

Comparison report

- Reference length: 15 nucleotides.
- Observed length: 15 nucleotides.
- Matching positions: 14.
- Mismatches: 1.
- Percentage of matching positions: 93.33%.
- Mismatch location: Position 7, reference G, observed C.

You could color-code the sequences to highlight mismatches.

What existing tools already do

BLAST identifies regions of sequence similarity, while Benchling provides more

comprehensive sequence analysis and alignment capabilities.

Your application would not offer their full functionality. It would instead provide a simple, transparent comparison workflow for short sequences.

What would make your version useful?

Focus on explaining discrepancies rather than merely reporting a similarity score.

For example:

- Show exactly where the sequences differ.
- Separate matches from mismatches visually.
- Explain what a nucleotide substitution is.
- Let users compare multiple observed sequences against the same reference.
- Produce a summary table that can be downloaded as a CSV file.

This last feature is particularly useful for demonstrating data processing.

The important limitation

For the initial version, require equal-length sequences and compare them position by position. If the lengths differ, explain that the basic comparison mode cannot reliably identify all corresponding positions.

Insertions and deletions require more sophisticated sequence alignment. You could investigate adding a basic alignment algorithm later, but it should not be necessary for the first version.

Also, a mismatch does not automatically mean a biological mutation occurred. It could

reflect a sequencing error, a sample difference, or another cause that requires investigation.

Technical difficulty: 3/5.

You would use string processing, loops, conditional statements, lists, and simple calculations. Those are excellent skills to demonstrate in a CS1 project.

Idea 2: AmpliconQC — Screening DNA sequences for potential quality issues

Best if you want to focus on quality control.

The scientific problem

Before researchers use DNA sequences in downstream analyses, they may need to check whether the data contains unexpected characters, unusually short sequences, ambiguous nucleotides, or other properties that warrant further review.

A collection of sequences makes these checks more tedious because the researcher must inspect multiple records.

Your solution

Build a tool that accepts several DNA sequences and produces a quality-control report.

For each sequence, your application could calculate:

Metric	What it tells the user

Sequence length	How many nucleotides are present
GC content	Percentage of nucleotides that are G or C
Ambiguous bases	How many positions contain symbols such as N
Invalid characters	Whether the input contains symbols outside the accepted alphabet
Repeated sequences	Whether multiple records contain identical sequences
Length outliers	Whether a sequence differs substantially from the collection's typical length

Your application could flag sequences for review rather than declaring them biologically invalid.

For example, if the user provides ten sequences and one contains unexpected characters, the report highlights that record.

What existing tools already do

FastQC and other sequencing-quality tools address quality assessment for sequencing

data, while other tools perform sequence preprocessing and filtering.

Your version would be much simpler. It would analyze the nucleotide strings themselves rather than claiming to evaluate raw sequencing quality.

A useful project distinction

Instead of trying to replicate a comprehensive sequencing-quality pipeline, focus on a quick screening report for a small collection of DNA sequences.

You could also explain why a particular property deserves review.

For example:

- A sequence containing many ambiguous bases might be difficult to interpret.
- A sequence much shorter than the others might deserve inspection.
- Identical sequences might be intentional or might indicate duplicate records.

These are flags for human review, not automatic proof of an error.

Technical difficulty: 3/5.

This idea is a good fit for lists, loops, dataclasses, input validation, statistics, and charts.

Idea 3: RestrictionMap — Understanding where DNA can be cut

Best if you want a tool that resembles something used in molecular biology laboratories.

The scientific problem

Researchers working with plasmids and other DNA molecules may need to determine where particular restriction enzymes recognize and cut a DNA sequence.

These enzymes recognize particular nucleotide patterns. Locating those patterns helps researchers understand the possible fragments produced by restriction digestion.

Your solution

Build a tool that scans a DNA sequence for recognition sites associated with a small, predefined collection of restriction enzymes.

For example, suppose a user enters a sequence and selects an enzyme that recognizes `GAATTC`.

Your program searches for every occurrence of that pattern and reports its position.

The user could then:

- View the locations of recognition sites.
- Compare the sites recognized by different enzymes.
- See how many sites occur in the sequence.
- Estimate fragment lengths under simplified digestion assumptions.
- Export a restriction-site report.

What existing tools do

The EMBOSS `restrict` and `remap` tools already identify restriction sites and can report their locations. Some options also report potential fragment lengths.

You would not be inventing this functionality. Instead, you could build a simpler, highly visual tool for exploring how recognition patterns relate to DNA sequences.

A possible differentiator

Create an interactive-looking sequence map that displays the DNA sequence and marks each recognition site.

You could also let the user select two enzymes and compare their recognition patterns.

Difficulty: 3/5. Searching for patterns and reporting positions is manageable with basic Python.

One limitation: real restriction digestion can depend on factors beyond the recognition sequence, including methylation and enzyme-specific behavior. Your initial version should clearly describe its simplified assumptions.

Idea 4: PrimerCheck — Comparing candidate PCR primers against a target sequence

Best if you want to focus on a specific molecular biology workflow.

The scientific problem

Researchers use PCR primers to target particular DNA regions. A primer that matches the wrong location or has unsuitable properties may not behave as intended.

Choosing appropriate primers and assessing their specificity are important parts of

experimental planning.

Your solution

Create a basic tool that evaluates candidate primer sequences against a user-provided DNA template.

Potential features:

- Validate primer sequences.
- Calculate primer length.
- Calculate GC percentage.
- Estimate melting temperature using a clearly documented simplified formula.
- Search for exact matches and reverse-complement matches within the template.
- Report candidate binding positions.
- Compare several candidate primers.

What existing tools do

The NCBI Primer-BLAST combines primer-design functionality with database-based specificity checking. It evaluates potential matches against selected biological databases and is substantially more sophisticated than a simple sequence scanner.

What your version could focus on

Rather than designing experimentally validated primers, you could create an educational primer inspection and reporting tool.

It would explain why each property matters and show where a candidate sequence matches the supplied template.

Difficulty: 3.5/5.

The basic calculations are manageable, but accurate primer evaluation involves more biology than GC content and exact matching. Your tool should not claim that a candidate is experimentally suitable or specific to an organism without appropriate analysis.

Idea 5: VariantViewer — Visualizing differences among DNA sequences

Best if you enjoy algorithms and data visualization.

The scientific problem

Researchers may need to compare multiple DNA sequences to investigate how they differ.

For example, a collection of sequences from different samples might contain substitutions at several positions. Finding those differences helps researchers describe variation within the collection.

Your solution

Create a tool that accepts a reference sequence and multiple observed sequences.

It could:

- Compare each sequence against the reference.
- Count matching and mismatching positions.
- Identify positions where the samples differ.
- Calculate the frequency of each nucleotide at a position.
- Generate a table of observed differences.

- Visualize sequence variation using a heatmap or another simple chart.

Example

Suppose three sample sequences differ from a reference at the same position.

Your tool could show that two samples contain `C` at position 25, while one contains `T`.

This is a description of the supplied sequences, not proof that the difference has a biological effect.

What existing tools do

BLAST focuses on identifying sequence similarities, while Benchling offers sequence alignment and visualization features.

Your project could focus specifically on making differences among a small collection of sequences easy to inspect.

For your initial version, require equal-length sequences and compare corresponding positions. A full alignment tool would require more sophisticated algorithms.

Difficulty: 3.5/5. This has more analytical depth than a basic sequence analyzer, especially if you add nucleotide-frequency calculations and visualization.

---

4. Which idea addresses the strongest specific problem?

Here is how I would compare them for your project.

Project	Specific scientific problem	Main technical challenge	Main limitation
SequenceCheck	Identifying discrepancies between expected and observed sequences	Comparing sequences and reporting mismatches	Basic version assumes equal lengths
AmpliconQC	Screening a collection of sequences for potential quality issues	Validation, statistics, and outlier detection	Cannot assess raw sequencing quality without quality data
RestrictionMap	Finding restriction-enzyme recognition sites	Pattern searching and position calculations	Limited enzyme collection
PrimerCheck	Inspecting candidate primers against a supplied template	Sequence matching and primer calculations	Cannot establish full primer specificity
VariantViewer	Summarizing variation across multiple sequences	Position-by-position comparison and visualization	Basic version does not handle insertions and deletions

All five are realistic, but I would prioritize the first, second, and fifth because they combine meaningful scientific data processing with features you can implement and test in Python.

5. My recommendation: Build SequenceCheck with a quality-control component

I would combine the most useful aspects of SequenceCheck and AmpliconQC into one focused application.

The project would not simply be a DNA analyzer. Its purpose would be:

Help researchers inspect a small collection of DNA sequences, identify potential data-quality issues, and compare observed sequences against an expected reference.

That gives you a specific workflow to investigate, a clear user, and measurable outputs.

You could call it SeqAudit: DNA Sequence Quality and Comparison Tool.

The problem statement

Here's the central problem I'd propose investigating:

Researchers who work with short DNA sequences may need to validate their input data and compare observed sequences against an expected reference. Performing these checks manually can be tedious, especially when several sequences must be reviewed. A lightweight tool that automatically checks sequence formatting, summarizes basic sequence properties, and highlights differences from a reference could make this initial inspection more convenient.

Notice the wording: you're proposing to make a workflow more convenient, not claiming that scientists lack existing software for it.

The target user

Your initial target user could be:

- An undergraduate biology student working with DNA sequence data.
- A researcher who needs to inspect a small set of sequences.
- A teaching laboratory instructor who wants students to explore sequence differences.
- A bioinformatics beginner learning how to interpret sequence-comparison results.

For a CS1 project, I would focus on students and researchers working with small sequence collections, rather than attempting to support large-scale sequencing pipelines.

The core workflow

Imagine a user has five DNA sequences and wants to compare them against a reference.

They would:

1. Paste the reference sequence.
2. Enter or upload the observed sequences.
3. Run basic input validation.
4. View sequence lengths, GC content, and ambiguous-base counts.
5. Compare equal-length sequences against the reference.
6. View the positions where mismatches occur.
7. Download a summary report.

This is a complete workflow with a clear beginning and end.

---

6. What would make SeqAudit technically interesting?

I would build it in three layers.

Layer 1: Sequence validation

The application checks whether the supplied sequences can be analyzed.

Possible checks include:

- Empty input.
- Invalid characters.
- Sequence length.
- Ambiguous nucleotide symbols.
- Duplicate sequences.

For example, if a user enters `ATGCXZ`, your program can report that `X` and `Z` are not accepted in the application's basic DNA alphabet.

You could optionally support `N`, which represents an unknown or ambiguous nucleotide, while explaining how it affects your calculations.

Layer 2: Sequence comparison

The user selects a reference sequence and compares one or more observed sequences against it.

For equal-length sequences, your program can calculate:

$$\text{Identity}(\%)=\text{Number of matching positions}\text{Sequence length}\times 100$$

$$\text{Identity}(\%)=$$

$$\frac{\text{Number of matching positions}}{\text{Sequence length}} \times 100$$

Your application would also report:

- The total number of matching positions.
- The number of mismatches.
- The mismatch positions.
- The reference and observed nucleotide at each mismatch.

For example, if 98 of 100 positions match, the positional identity is 98%.

This simple measure is appropriate for equal-length, position-by-position comparisons. It is not a substitute for general sequence alignment, and it should not be used to infer biological relatedness on its own.

Layer 3: Results and visualization

Display the results in a scientific dashboard.

Potential visualizations include:

- A bar chart of sequence lengths.
- A bar chart of GC content.
- A table highlighting mismatch positions.
- A summary of how many sequences passed each validation check.

You could use Matplotlib if it works in your Drafter environment.

This layer would make the project visually impressive without requiring complicated biological algorithms.

---

7. How you could satisfy the project rubric

Your application naturally supports the required dataclasses and state fields.

Dataclass 1: `DNASequence`

Field	Purpose
`name`	Identifies the sequence
`sequence`	Stores the nucleotide string
`length`	Stores its length
`gc_content`	Stores its calculated GC percentage
`ambiguous_count`	Stores the number of ambiguous bases

Dataclass 2: `SequenceComparison`

Field	Purpose
`reference_name`	Identifies the reference
`sample_name`	Identifies the observed sequence
`match_count`	Number of matching positions
`mismatch_count`	Number of mismatching positions
`identity_percentage`	Percentage of matching positions

Suggested `State` fields

- `sequences`: a list of `DNASequence` objects.
- `comparisons`: a list of `SequenceComparison` objects.
- `reference_sequence`: the currently selected reference.
- `analysis_count`: number of analyses performed.
- `filter_mode`: the currently selected result filter.

You would need to ensure that at least four state fields are meaningfully modified by routes.

For example, adding sequences changes `sequences`, comparing sequences adds to `comparisons`, selecting a reference changes `reference_sequence`, and running an analysis updates `analysis_count`.

Suggested routes

Route	Purpose
`/`	Project overview and dashboard
`/input`	Enter a DNA sequence
`/collection`	View stored sequences
`/quality-check`	Validate sequences and calculate statistics
`/compare`	Select sequences to compare
`/comparison-results`	Display comparison results
`/visualization`	Display charts and summary statistics

`/history`	Review previous analyses

You could implement all of these with Drafter, lists, dataclasses, and functions. You would not need a database or external API for the core functionality.

---

8. One final consideration: What does your tool do better than existing software?

This is the most important question to investigate before committing to the idea.

Benchling, BLAST, and other bioinformatics tools already perform sophisticated sequence analysis. Your project should not claim that your application is more accurate or more powerful.

Instead, identify a narrower value proposition.

For example:

SeqAudit is a lightweight educational tool that combines basic DNA sequence validation, reference comparison, and visual reporting in a single, easy-to-understand workflow for small sequence collections.

You can investigate whether this combination is useful for your intended users. You do not need to prove that no existing software has the same features; you need to explain why your particular scope is worthwhile.

To support your planning document, I would research three questions:

1. What sequence checks do students or researchers commonly perform before interpreting DNA data?
2. What do existing tools require users to enter, and how do they display their results?
3. Would combining basic validation, comparison, and visual reporting make the workflow easier for a beginner?

For your research, start with the official resources for NCBI BLAST, Benchling, and FastQC.

My verdict: SeqAudit is a strong candidate if you want a project that feels like a real bioinformatics utility, demonstrates actual algorithms, and remains achievable within your course constraints.

The key is to make the comparison and quality-control workflow your central feature, rather than adding lots of unrelated DNA tools.

i like idea 3 better

I think RestrictionMap is an excellent choice if you want to build something that feels like a real molecular biology tool. It addresses a specific laboratory task, involves genuine biological concepts, and gives you plenty of opportunities to demonstrate technical programming skills.

Let's develop the idea into a project that is both useful and realistic for your CS1 final.

🧬 Your Project: RestrictionMap

An interactive DNA restriction-site analysis tool

1. The problem you're solving

What problem do scientists face?

Molecular biologists often use restriction enzymes to cut DNA at particular nucleotide sequences. Researchers working with plasmids or other DNA fragments may need to determine which restriction sites are present and what DNA fragments could result from cutting at those sites.

Doing this manually can become tedious when a sequence contains many nucleotides or when several enzymes need to be compared.

For example, a researcher might want to answer:

- Does this DNA sequence contain a recognition site for a particular restriction enzyme?
- Where along the sequence are the sites located?
- How many times does each enzyme recognize the sequence?
- If the DNA is cut at these sites, what fragment lengths are expected?
- Which of several enzymes produces a particular pattern of fragments?

Your application would automate these calculations and display the results in an easy-to-understand format.

Your core problem statement could be:

Researchers and biology students sometimes need to inspect DNA sequences to determine where restriction enzymes recognize and cut the DNA. RestrictionMap will automate restriction-site detection and predicted fragment-length calculations, helping users explore

DNA sequences without manually searching for recognition patterns.

This is a legitimate scientific computing problem, and you can implement its basic functionality using Python string processing.

---

2. What existing tools already do

Before deciding what makes your application useful, you should understand the existing tools.

Tool 1: EMBOSS Restrict

Official resource: EMBOSS Restrict documentation

EMBOSS Restrict identifies restriction enzyme recognition sites in DNA sequences. It can report their positions and information about the resulting fragments.

What this means for your project: You don't need to invent restriction-site detection. You can create a simpler interface that makes the results easier for a beginner to understand.

Tool 2: Benchling

Official website: Benchling Molecular Biology

Benchling provides a broader molecular biology environment, including sequence analysis and visualization features.

What this means for your project: Your application can focus on one workflow rather than trying to reproduce a complete research platform.

Tool 3: NEBcutter

Official website: NEBcutter

NEBcutter is a tool for analyzing DNA sequences and identifying restriction enzyme cleavage sites.

What this means for your project: RestrictionMap would be entering an established category of bioinformatics software. Your differentiator should be its focused educational design and clear explanation of the calculations, rather than claiming to have more comprehensive analysis capabilities.

Your potential niche

I would position RestrictionMap as:

A lightweight, educational restriction analysis tool that lets users explore enzyme recognition sites, compare predicted digests, and understand how DNA fragments are calculated.

That's a realistic project goal.

---

3. How the application would work

Imagine a user has a DNA sequence and wants to investigate how two different restriction enzymes would cut it.

They open RestrictionMap and follow this workflow:

Step 1: Enter DNA

The user pastes a sequence into a text area.

For example:

```
ATGGAATTCGCTGAATTCACG
```

Step 2: Select an enzyme

The application offers a small predefined collection of enzymes and their recognition sequences.

For example, EcoRI recognizes:

```
GAATTC
```

Step 3: Analyze the sequence

The application searches for occurrences of the recognition pattern and records their positions.

Step 4: Calculate fragment lengths

The application uses the positions of the cuts to calculate the lengths of the resulting DNA fragments.

Step 5: Display the results

The user sees:

- The selected enzyme and its recognition sequence.
- The number of recognition sites.
- The positions of those sites.
- The predicted fragment lengths.
- A visual representation of the sequence and cut positions.

Step 6: Compare enzymes

The user selects a second enzyme and compares the results.

This is the core application. Everything else should build on it.

---

4. The features I would build

I'd organize the project into three levels so that you can get a working version running before attempting the more advanced features.

Level 1: Essential features

These are the features I would consider mandatory for your own development plan.

A. DNA sequence input and validation

Users can paste a DNA sequence into a `TextArea`.

Your program:

- Removes whitespace and line breaks.
- Converts letters to uppercase.
- Checks for invalid characters.
- Rejects empty input.
- Stores the sequence for analysis.

You could also allow users to enter a sequence name.

B. Restriction enzyme selection

Create a small predefined collection of enzymes.

For example:

Enzyme	Recognition sequence
EcoRI	GAATTC
BamHI	GGATCC
HindIII	AAGCTT
HaeIII	GGCC

These are examples of commonly used restriction enzymes and their recognition

sequences.

You do not need hundreds of enzymes. Start with four to six well-understood examples.

C. Restriction-site detection

Search the DNA sequence for every occurrence of the selected recognition pattern.

For example, if the DNA sequence contains two occurrences of `GAATTC`, your program should report both.

For each occurrence, record its starting position.

Be consistent about whether positions start at zero or one. For a user-facing biology tool, one-based positions are generally easier to understand.

D. Predicted fragment lengths

For a simple linear DNA molecule, calculate the distances between consecutive cut positions, including the beginning and end of the sequence.

Suppose a hypothetical DNA molecule is 1,000 base pairs long and has cut positions at 200 and 700.

The predicted fragment lengths would be:

- 200 bp
- 500 bp
- 300 bp

The lengths should add up to the original sequence length.

That gives you a useful correctness check for your code.

Important biological detail: Restriction recognition sites and cleavage positions are not always interchangeable. For a correct fragment calculation, your implementation must account for where each enzyme actually cuts relative to its recognition sequence. For the initial version, choose enzymes with well-defined, straightforward cleavage patterns and document your assumptions.

Level 2: Features that make it impressive

Once the essential features work, consider these additions.

E. Compare two enzymes

Let users analyze the same DNA sequence using two different enzymes.

Show a comparison table:

Measurement	Enzyme A	Enzyme B
Recognition sites	2	1
Predicted fragments	3	2

Smallest fragment	150 bp	350 bp
Largest fragment	500 bp	650 bp

The values above are illustrative rather than experimental results.

This feature involves useful data processing and gives your application a clear purpose beyond displaying a single result.

F. Visual restriction map

Display a horizontal representation of the DNA molecule, with marks showing the positions of restriction sites.

For example:

```
DNA sequence: 1,000 bp

0 bp        200 bp                 700 bp       1,000 bp
 |-------------|----------------------|-------------|

              EcoRI                  EcoRI
```

You can initially render a simple textual map using Drafter components. If time permits, you could use Matplotlib or another supported plotting approach to produce a more polished diagram.

G. Saved analyses

Let users save their analyses within the application and review previous results.

This feature naturally supports your required list of dataclass instances.

If you want results to persist between visits, investigate browser local storage. Otherwise, saved analyses can remain available for the current application session.

H. Downloadable report

Generate a text or CSV report containing:

- The DNA sequence name.
- The selected enzyme.
- Recognition sequence.
- Recognition-site positions.
- Predicted fragment lengths.

Drafter's supported download capabilities would determine the simplest implementation. You can leave this feature until after the main workflow works.

---

5. How to structure the Python program

Your rubric requires at least two dataclasses, including a collection of instances in your application state.

RestrictionMap fits this requirement naturally.

Dataclass 1: `DNASequence`

This represents the DNA molecule being analyzed.

Field	Type	Purpose
`name`	`str`	User-provided sequence name
`sequence`	`str`	DNA nucleotide sequence
`length`	`int`	Number of nucleotides
`description`	`str`	Optional notes about the sequence

Dataclass 2: `RestrictionAnalysis`

This represents the results of analyzing one sequence with one enzyme.

Field	Type	Purpose
`sequence_name`	`str`	Name of the analyzed DNA sequence
`enzyme_name`	`str`	Selected restriction enzyme
`recognition_sequence`	`str`	Nucleotide pattern recognized by the enzyme

`site_positions`	`list[int]`	Positions of detected recognition sites
`fragment_lengths`	`list[int]`	Calculated DNA fragment lengths

You can add fields later if needed, but these are enough to support the main workflow.

Your `State` dataclass

I would consider these fields:

Field	How it changes
`sequences`	Add new DNA sequences
`analyses`	Save new restriction analysis results
`selected_sequence`	Change the sequence being analyzed
`selected_enzyme`	Change the selected enzyme
`comparison_mode`	Enable or disable enzyme comparison

The first four can be meaningfully modified by different routes. That gives you room to meet the state requirements.

For example:

- The sequence-input route adds to `sequences`.
- The analysis route adds to `analyses`.
- Selecting a sequence updates `selected_sequence`.
- Selecting an enzyme updates `selected_enzyme`.

Make sure these are actual state changes, not just values that are initialized and never updated.

---

6. Your proposed website structure

You need at least seven routes and five pages. Here's a structure that naturally satisfies those requirements.

Route	Purpose
/	Homepage and application overview
`/input`	Enter a DNA sequence
`/sequences`	View stored DNA sequences

`/select-enzyme`	Select a restriction enzyme
`/analyze`	Run restriction-site analysis
`/results`	View recognition sites and fragment lengths
`/compare`	Compare two enzyme analyses
`/saved-analyses`	Review saved results
`/about`	Explain the biology and model assumptions

That's nine routes.

Some routes can share visual components, but I'd make sure at least five of them have clearly distinct page designs or purposes.

You can implement the main interface using Drafter components such as buttons, text boxes, text areas, select boxes, and checkboxes.

---

7. How this project meets your rubric

Requirement	Planned implementation
7+ routes	Nine routes
5+ pages	Homepage, Sequence Input, Sequence Library, Enzyme Selection, Results, Comparison, and Saved Analyses
`State` with at least 4 fields	Five meaningful state fields
Second dataclass	`DNASequence` and `RestrictionAnalysis`
Four modified state fields	Sequences, analyses, selected sequence, selected enzyme
Three input fields	Sequence name, DNA sequence, enzyme selection, plus optional comparison controls
Three meaningful `if` statements	Empty-sequence validation, invalid-character detection, and checking whether recognition sites exist
Meaningful loop	Iterate through sequences or recognition sites to display results and calculate fragment lengths

Legitimate purpose	Analyze DNA restriction sites and predicted fragments
No global variables	Keep changing application data in `State`
Drafter library	Use Drafter routes and components
Advanced techniques documented	Explain any algorithms beyond course material in comments or docstrings

You'll also need to write unit tests for your functions where possible and test your routes. You can test the underlying sequence-analysis functions independently of the interface, which should make debugging easier.

---

8. What could make RestrictionMap stand out?

If you want this to look like a genuine scientific application rather than a basic class assignment, I would prioritize three things.

1. Correctness

Make sure recognition sites are detected accurately and fragment lengths are calculated according to documented biological assumptions.

2. Clear visualizations

Give users a sequence map, a list of cut sites, and a fragment-length table. These outputs make the results easier to interpret.

3. Scientific explanations

For each analysis, explain what the output means and what the application does not establish.

For example, identifying a restriction site does not establish that an enzyme will necessarily cut the DNA under a particular laboratory protocol. Actual digestion depends on experimental conditions and other factors.

These explanations will also help you demonstrate that you understand the generated code and the biology behind it.

---

9. Your next step: Define the exact scope

Before you complete your project-pitch worksheet, I recommend making one decision.

Which version of RestrictionMap do you want to build?

A. Restriction Site Finder — Moderate difficulty

Users paste a DNA sequence, choose an enzyme, and see where its recognition sites occur. The application calculates predicted fragment lengths for a simple linear DNA

molecule.

B. RestrictionMap Lab — Challenging but manageable

Everything in A, plus a visual DNA map, comparison of two enzymes, and a saved-analysis history.

C. RestrictionMap Explorer — More ambitious

Everything in B, plus circular plasmid support, more detailed enzyme-cutting behavior, and downloadable reports.

My recommendation is B. It provides enough technical depth to stand out, naturally satisfies the assignment requirements, and remains realistic for a three-week project.

I'd build the sequence analysis and fragment calculations first, then add the visual map and comparison features. That way, you will have a working scientific application even if you run out of time for the more advanced features.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.